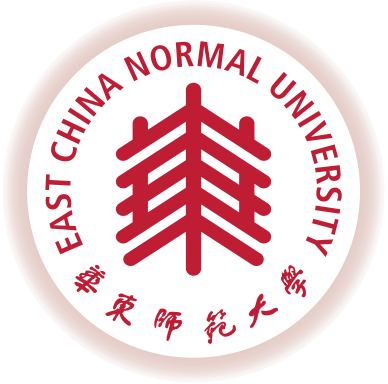
2026 届本科生学士学位论文　　学校代码：10269



華東師範大學

East China Normal University

本科生毕业论文

基于 Peach 模糊测试框架的协议语义感知变异增强方法的研究与实现

Research and Implementation of a Protocol Semantic-Aware Mutation Enhancement Method Based on the Peach Fuzzer

姓　　名：李鹏达

学　　号：10225101460

学　　院：软件工程学院

专　　业：软件工程

指导教师：苏亭

职　　称：教授

2026 年 4 月

华东师范大学学位论文诚信承诺

本毕业论文是本人在导师指导下独立完成的，内容真实、可靠。本人在撰写毕业论文过程中不存在请人代写、抄袭或者剽窃他人作品、伪造或者篡改数据以及其他学位论文作假行为。

本人清楚知道学位论文作假行为将会导致行为人受到不授予/撤销学位、开除学籍等处理（处分）决定。本人如果被查证在撰写本毕业论文过程中存在学位论文作假行为，愿意接受学校依法作出的处理（处分）决定。

承诺人签名：李鹏飞　　日期：2026 年 5 月 10 日

华东师范大学学位论文使用授权说明

本论文的研究成果归华东师范大学所有，本论文的研究内容不得以其它单位的名义发表。本学位论文作者和指导教师完全了解华东师范大学有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权华东师范大学可以将论文的全部或部分内容编入有关数据库进行检索、交流，可以采用影印、缩印或其他复制手段保存论文和汇编本学位论文。

保密的毕业论文（设计）在解密后应遵守此规定。

作者签名：李鹏飞　导师签名：　　日期：2026 年 5 月 10 日

目录

摘要 . . . . . **I**
ABSTRACT . . . . . **II**
1、 绪论 **1**
1.1 研究背景与意义 1
1.2 国内外研究现状 2
1.3 主要研究内容 4
1.4 论文组织架构 4
2、 相关知识 **6**
2.1 网络协议 6
2.2 模糊测试与协议模糊测试 8
2.3 Peach 模糊测试框架 10
2.4 大模型智能体 15
3、 协议语义感知变异增强方法 **17**
3.1 方法概述 17
3.2 大模型智能体驱动的测试组件生成流水线 17
3.3 两阶段变异策略 28
3.4 LLMPeach 的实现 29
4、 实验评估与分析 **32**
4.1 实验设置 32
4.2 RQ1: 分支覆盖 34
4.3 RQ2: 独特分支 37
4.4 RQ3: 两阶段变异策略 39
5、 总结与展望 **42**
参考文献 **43**
附录 **45**
致谢 **46**

基于 Peach 模糊测试框架的协议语义感知变异增强方法的研究与实现

摘要：

针对物联网与工业互联网中复杂网络协议的漏洞挖掘需求，本文聚焦传统 Peach 模糊测试框架因语义感知能力不足导致变异盲目、难以触发深层逻辑的问题，提出一种协议语义感知变异增强方法，并将其集成于 Peach 框架中，开发了 LLMPeach 模糊测试框架。

该方法通过设计自动化智能体流程引导大语言模型深度解析协议规范文档，自主提取字段含义、取值约束及逻辑关联，构建数据模型，并生成语义相关的变异器与修复器。在此基础上，构建两阶段变异策略——第一阶段通过大模型生成的变异器对协议字段进行语义关联的变异，第二阶段辅以随机变异以保障样本多样性。

在 5 个现实世界协议实现上的实验结果表明，相较于原始 Peach，LLMPeach 在被测协议实现代码分支覆盖上平均提高了 7.33%。研究证明，结合大语言模型的语义感知机制能有效弥补传统模糊测试在语义维度的缺陷，提升复杂网络协议的漏洞挖掘效率。

关键词：协议模糊测试，大语言模型，Peach 模糊测试框架

Research and Implementation of a Protocol Semantic-Aware Mutation Enhancement Method Based on the Peach Fuzzer

Abstract：

To address the vulnerability discovery needs of complex network protocols in the IoT and Industrial Internet, this paper focuses on the inherent limitations of the traditional Peach fuzzer–specifically its lack of semantic awareness, which leads to blind mutations and difficulty in triggering deep-seated logic. We propose an protocol semantic-aware mutation enhancement method and integrate it into the Peach fuzzer, developing the LLMPeach fuzzer.

This approach utilizes an automated Agent workflow to guide a Large Language Model in deeply parsing protocol specification documents. This allows for the autonomous extraction of field meanings, value constraints, and logical correlations to construct a DataModel and generate semantically relevant Mutators and Fixers. Based on this, a two-stage mutation strategy is constructed: the first stage uses LLM-generated mutators for semantically correlated mutations, while the second stage incorporates random mutations to ensure sample diversity.

Experimental results in 5 real-world protocol implementations demonstrate that, compared to the original Peach fuzzer, LLMPeach achieves an average increase of 7.33% in code branch coverage for the tested protocol implementations. This research proves that integrating the semantic awareness mechanism of LLMs effectively compensates for the semantic deficiencies of traditional fuzzing, thereby enhancing the efficiency of vulnerability discovery in complex network protocols.

Keywords： Protocol Fuzzing, Large Language Models, Peach Fuzzer

1、 绪论

1.1 研究背景与意义

随着互联网技术、物联网以及工业互联网的快速发展，网络协议已广泛应用于信息系统、嵌入式设备和关键基础设施之中。协议作为系统之间进行信息交互和协同工作的基础，其实现的正确性和健壮性直接关系到系统的安全性与可靠性。在实际应用中，协议实现往往涉及复杂的数据结构、严格的字段约束以及多阶段的状态转换逻辑，一旦在设计或实现过程中存在疏漏，极易引入安全缺陷。攻击者可以通过构造异常报文触发程序中的边界条件错误或逻辑异常，从而造成服务拒绝、系统崩溃甚至更为严重的安全后果[1-2]。一个典型的例子是 2014 年爆发的 Heartbleed 漏洞[3]，该漏洞源于 OpenSSL 实现 TLS/DTLS 协议心跳机制时边界检查缺失，导致攻击者能够远程泄露服务器内存中的敏感信息，造成了广泛的安全影响。

为了保障协议实现的安全性，研究人员和安全从业者一直在探索有效的测试方法来发现潜在的漏洞。模糊测试（Fuzzing）[2,4] 是一种自动化的软件测试技术，其核心在于向计算机程序提供无效、非预期或随机的数据作为输入，以触发异常行为并发现潜在缺陷。由于其无需完整源码支持、对测试目标适应性强，模糊测试在协议安全测试领域得到了广泛应用[5-6]。在现有的模糊测试工具中，Peach[7] 是一个具有代表性的经典框架，其通过数据模型、状态模型、变异器和变异策略的组合，为复杂协议的自动化测试提供了良好的工程化基础，至今仍在工业界广泛应用。

Peach 框架提供了一套结构化的协议建模语言，允许用户定义协议字段的类型、长度、取值范围以及状态机等信息，以指导测试用例的生成和变异，其性能也在很大程度上受协议模型的构建质量影响[8]。然而，从协议规范文档中提取有效信息以构建高质量的协议模型是一项复杂而耗时的任务，难以自动化完成，依赖测试人员手工编写，尤其对于状态复杂、约束严格的协议而言更是如此。而且，尽管可以定义出详细的数据模型和状态模型，在实际测试过程中，Peach 框架中的变异算子仍主要是字段类型相关的随机变异器，缺乏对特定协议语义的理解和利用。此外，其结构化建模对于协议的字段间约束表达能力有限，只能描述例如长度约束、数量约束等基本关系，对字段之间更为复杂的语义关联和逻辑关系难以表达和利用。这虽然能够基本保证测试用例在格式上合法，但随机变异往往破坏协议内部的语义一致性，导致大量生成的测试用例在解析或状态校验阶段即被丢弃，难以触发深层逻辑路径，制约了漏洞发现能力。

近年来，大语言模型（LLM）的兴起与飞速发展为协议模糊测试领域带来了新的机遇。将大语言模型引入协议模糊测试过程[9-10]，利用其在自然语言理解和知识抽取方面的显著优势，从协议规范文档中学习隐含规则和语义关系，参与变异逻辑的生

成，有望在保持输入结构合理性的同时，更有针对性地对字段语义进行探索。

在此背景下，研究一种基于 Peach 模糊测试框架的协议语义感知变异增强方法，具有重要的理论意义和实践价值。该研究有助于弥补传统模糊测试框架在语义层面的不足，推动模糊测试技术向更加智能和高效的方向发展，同时也为复杂网络协议实现的安全测试提供新的思路。

1.2 国内外研究现状

1.2.1 协议模糊测试

网络协议模糊测试作为一种自动化挖掘协议实现漏洞的重要技术，其核心挑战在于协议数据具有高度结构化语义，并且协议交互过程通常伴随复杂的状态依赖关系。因此，如何在模糊测试过程中同时实现协议语义理解与状态空间探索，逐渐成为该领域研究的核心问题。

现有协议模糊测试技术总体上可以分为两类：基于生成（Generation-based）的协议模糊测试与基于变异（Mutation-based）的协议模糊测试。前者依赖协议模型生成结构大体合法的测试报文，更关注协议语义与格式约束；后者则以已有样本为基础，通过不断变异输入，以提升状态空间探索能力。

在基于生成的协议模糊测试方向中，研究重点主要集中于协议建模与结构化测试数据生成。Peach[7] 是典型的基于规约的生成式模糊测试框架。它通过 XML 格式的“Peach Pit”文件定义协议的数据模型和状态模型，并依据字段类型、长度约束以及校验和逻辑生成符合协议语义的数据报文，从而有效绕过协议解析器的初步合法性检查。相比传统随机测试方法，Peach 能够生成更加结构化且有效的测试输入。然而，该工具高度依赖专家手工构建协议模型，在处理复杂协议或私有协议时存在较高的建模成本。

Boofuzz[11] 是当前开源社区中广泛使用的另一种生成式协议测试框架。与 Peach 基于 XML 的描述方式不同，Boofuzz 采用 Python 脚本定义协议结构，并提供了较为完善的会话管理、故障恢复与监控机制。Boofuzz 能够通过维护请求与响应之间的关联关系，对协议交互流程进行测试，因此在工程实践中具有较强的灵活性与可扩展性。然而，由于其测试过程仍主要依赖预定义结构与随机字段变异，缺乏覆盖率反馈引导机制，因此在复杂协议的深层状态探索方面仍存在一定局限。

为了进一步提升生成式协议模糊测试的探索能力，研究者开始尝试将覆盖率反馈机制引入生成式框架。Peach*[12] 在 Peach 基础上进行了改进，针对工业控制协议（ICS）的特点，引入了覆盖率引导的报文拆解与重组机制。与传统 Peach 依赖固定变异规则不同，Peach* 能够动态收集执行路径反馈，并对触发新状态的优质报文进行碎片化重组，从而生成更具探索性的测试用例。

相比于生成式方法，基于变异的协议模糊测试更加关注状态空间覆盖与执行路径探索。AFLNet[13] 是该方向中的代表性工作，其首次系统性地将覆盖率引导思想引入网络协议模糊测试。AFLNet 通过监控服务器返回的响应码（如 FTP 的 220 或 SMTP 的 250）推断协议有限状态机（FSM），并依据状态反馈动态调整种子选择策略，从而优先探索尚未到达的协议状态。相比传统模糊测试工具，AFLNet 在有状态协议中的路径覆盖能力得到了显著提升。

在 AFLNet 的基础上，研究者进一步关注协议消息之间的语义依赖关系。SGFuzz[14] 提出了基于状态图引导的变异策略，通过自动构建协议状态图识别消息序列中的依赖关系。相较于 AFLNet 中随机性较强的序列变异方式，SGFuzz 能够生成更加符合协议逻辑的消息链，从而减少因违反协议规约而产生的大量无效测试用例。

研究者还关注真实网络环境中的反馈稳定性问题。NSFuzz[15] 针对网络延迟、丢包以及超时等因素导致的反馈噪声，引入了细粒度的网络状态感知机制，通过优化同步策略与超时处理降低网络不确定性对模糊测试的影响。此外，NSFuzz 还增强了对协议报文字段结构的理解能力，实现针对复杂字段的精准变异，从而提升了真实网络环境中的测试鲁棒性。

1.2.2 大语言模型辅助的协议模糊测试

越来越多的研究开始探索利用大语言模型来增强协议模糊测试。一些工作尝试使用大语言模型来丰富用于模糊测试的初始种子库，或是从协议规范文档中自动提取协议字段和状态机信息以构建更为准确的协议模型。

其中，最具有代表性的是 ChatAFL[9]。它基于 AFLNet 构建，通过与大语言模型交互提取协议的机器可读语法，从而实现结构感知的变异操作。同时，该方法利用模型生成多样化的消息以扩展初始种子语料库。在模糊测试陷入覆盖率瓶颈时，ChatAFL 进一步通过预测交互序列中的下一条消息，引导测试向新的状态空间探索。

BSFuzzer[16] 则使用大语言模型进行语义推理与行为验证。该工具面向 BLE 协议场景，利用大语言模型从 Bluetooth 规范中自动提取协议状态机以及报文级语义信息，并在此基础上生成具有上下文感知能力的测试序列。与传统基于随机变异的方法不同，BSFuzzer 不仅能够生成具有语义意义的字段变异，还能够主动构造违反状态迁移规则的异常交互行为，并将其组织为多步消息序列。此外，该方法进一步利用大语言模型对设备响应进行语义一致性分析，从协议行为层面判断目标实现是否符合规范预期，从而提升了对逻辑漏洞与协议实现缺陷的检测能力。

相比于依赖协议规范进行语义建模的方法，MSFuzz[17] 更进一步地关注协议实现源码中隐含的消息语法知识挖掘。该方法首先通过预处理技术定位关键的消息解析逻辑，随后借助大语言模型构建细粒度的消息语法树，以识别消息类型、字段约束关

系以及合法取值范围等深层语义信息。在模糊测试阶段，MSFuzz 利用提取出的语法知识对初始种子语料库进行扩展，并实现语法感知的变异策略，从而生成更容易通过协议语法检查的高质量测试用例。相较于仅基于协议文档进行建模的方法，MSFuzz 能够更加贴近真实协议实现细节，增强了模糊测试对复杂解析逻辑的探索能力。

纵观协议模糊测试的发展历程，尽管研究重点已从初期的随机变异演进至如今的状态感知与语义启发，但在理解协议字段的深度语义，以及针对这些语义信息进行更有针对性的探索方面，仍存在提升空间。此外，目前学术界的研究重心大多倾向于对以 AFLNet 为代表的基于变异的模糊测试框架进行增强，而对于 Peach 模糊测试框架，相关研究相对较少，且大多集中在为其添加覆盖率反馈上[12,18]。利用大语言模型的知识抽取与推理能力来解决 Peach 框架的现有问题，仍是一个尚未充分探索的研究方向。

1.3 主要研究内容

针对传统 Peach 模糊测试框架协议建模成本高、变异算子缺乏语义感知能力及难以覆盖深层逻辑路径等问题，本文研究并实现了一种大语言模型智能体驱动的协议语义感知变异方法。该方法利用大语言模型的知识抽取与推理能力生成代码，集成到 Peach 的变异流程中，旨在提升测试用例的质量与漏洞发现效率。本文的主要研究内容包括：

1) 设计并实现了一个自动化智能体（Agent）流程，指导大语言模型从协议规范文档中理解语义信息，生成包括数据模型、变异器和修复器在内的协议测试组件；
2) 设计并实现了与测试组件适配的两阶段变异策略；
3) 将上述方法集成到 Peach 模糊测试框架中，形成 LLMPeach，并在多个现实世界协议实现上进行了实验评估与分析。

1.4 论文组织架构

第一章　绪论

绪论部分介绍了研究的背景与意义，分析了当前协议模糊测试领域的核心瓶颈与挑战，并概述了国内外在协议模糊测试及大语言模型辅助的协议模糊测试方面的研究现状。最后，明确了本文的主要研究内容和论文的组织架构。

第二章　相关知识

相关知识部分详细介绍了模糊测试的基本概念、Peach 模糊测试框架的核心架构与工作流程，以及大语言模型智能体的原理与应用，为后续章节的研究内容奠定理论基础。

第三章　协议语义感知变异增强方法

协议语义感知变异增强方法部份详细阐述了本文提出的方法的设计，包括大模型智能体驱动的测试组件生成流水线和两阶段变异策略，并介绍了 LLMPeach 框架的整体架构与实现细节。

第四章　实验评估与分析

实验评估与分析部分通过在多个现实世界协议实现上的实验，评估了所提出的协议语义感知变异增强方法在代码分支覆盖、独特分支等方面的表现，并与基线 Peach 框架进行了对比分析。同时，通过消融研究验证了各个组件对整体性能提升的贡献。

第五章　总结与展望

总结与展望部分回顾了本文的主要研究工作和贡献，分析了研究中存在的不足，并提出了未来可能的研究方向和改进措施。

2、 相关知识

2.1 网络协议

网络协议是分布式系统与通信网络的基础。为了使网络中不同体系结构的计算机能够准确地交换数据，必须对数据的传输格式、传输速率、步骤等制定统一的规则。这些为进行网络数据交换而建立的规则、标准或约定，统称为网络协议[19]。

2.1.1 协议报文的基本要素

一个严谨的协议报文定义通常由语法、语义和时序三个基本要素构成。其中，语法（Syntax）定义了数据与控制信息的结构或格式，包括数据字段的类型、长度、先后顺序以及特定的标识符；语义（Semantics）规定了数据每一部分的具体含义，例如某个字节代表的是“读请求”还是“写请求”，以及如何解释复杂的复合字段；时序（Timing）则指明了实体间通信的顺序，以及由于事件触发而导致的状态演变，即协议状态机。这三个要素共同构成了协议的完整定义，确保了通信双方能够正确理解和处理交换的数据。

2.1.2 协议的分类

根据数据封装与语义解析方式的不同，网络协议主要可分为文本协议（Text-based Protocol）与二进制协议 (Binary Protocol) 两大类。文本协议（如 HTTP、SMTP）使用可读的文本格式进行通信，便于调试和分析；而二进制协议（如 MQTT、DTLS）则使用紧凑的二进制格式，通常具有更高的效率和更小的消息体积。

2.1.3 协议的交互

网络协议的交互过程主要基于经典的客户端-服务器（Client-Server, C-S）架构。该架构将通信双方划分为两个具有明确职能差异的角色：服务器（Server）与客户端（Client）。服务器作为服务的提供方，通常运行在高性能主机上并处于长期的监听状态，负责管理核心资源并被动等待接入；客户端则作为服务的请求方，根据用户需求主动向服务器发起连接。每一轮交互始于客户端发送的请求报文，服务器在接收并处理该报文后，返回相应的响应报文以完成闭环。在这种架构下，协议的交互具有明显的非对称性：交互的节奏和时机通常由客户端驱动，而服务器则负责处理并发请求并维持服务的稳定性。这种角色分明的设计简化了网络通信的组织形式，使得协议能够高效地实现资源共享与数据交换。

2.1.4 协议的特点

网络协议的核心特性主要体现在结构化的数据表达和严格的行为逻辑两个方面。

首先，在数据表达层面，协议通过定义清晰的报文结构来规定数据的静态布局。各字段具有固定的偏移和边界，从而保证信息交换的确定性。在此基础上，协议进一

步通过字段语义约束限定数据项的取值范围，并通过字段之间的依赖关系（如长度字段与校验字段）确保报文整体的一致性与完整性。

其次，在行为逻辑层面，协议具有明显的状态相关性。协议通常通过状态机（State Machine）来描述通信过程中的状态转换，只有在满足特定条件时才会发生状态迁移，从而明确各阶段的转换条件与执行规则，保证通信过程在时序和逻辑上的严谨性。

以图 2-1 和图 2-2 所示的 MQTT 协议控制报文为例，其具体的字段布局与状态迁移直观地反映了上述特性。在静态结构上，报文由固定报头中的 TYPE 和 FLAG 起始，紧随其后的是 REMAINING LENGTH 字段、VARIABLE HEADER 和 PAYLOAD。每个字段的长度和取值范围都收到了严格的定义和约束。例如，CONNECT FLAG 字段共包含 8 位，每一位都具有特定的语义含义，如第 0 位表示是否使用用户名，第 1 位表示是否使用密码等。此外，这些字段之间还存在相互依赖关系，如 REMAINING LENGTH 字段必须正确反映后续字段的总长度，CONNECT FLAG 字段中的特定位决定了 PAYLOAD 中是否包含用户名和密码等信息。

在动态行为上，MQTT Broker 的运行严格受控于状态机的变迁：设备初始处于 DISCONNECTED 状态，必须通过特定的连接序列才能进入 CONNECTING 并最终达到 CONNECTED 状态。如果在任何阶段出现不符合协议规范的报文，设备将立即返回错误响应并断开连接。

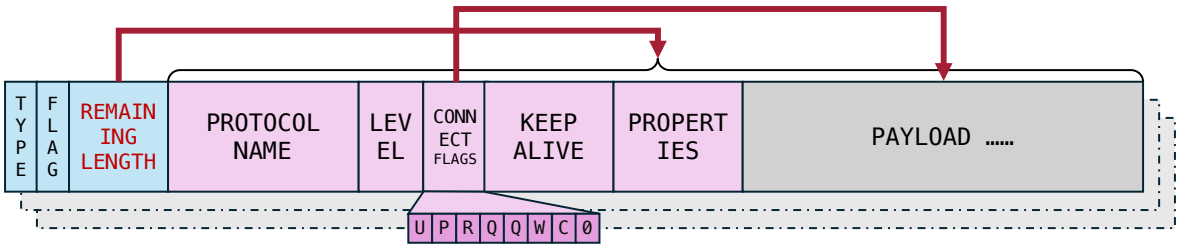


图 2-1　MQTT 请求消息结构示例

Figure 2-1　Example of MQTT Request Message Structure

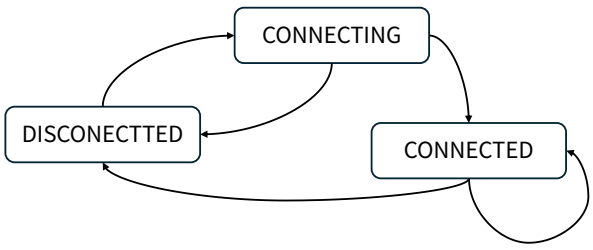


图 2-2　MQTT 协议 Broker 简易状态机示例

Figure 2-2　Example of MQTT Broker State Machine

2.2 模糊测试与协议模糊测试

模糊测试（Fuzzing）是一种通过向目标程序提供非预期的、随机的或构造的畸形输入，并监测程序是否出现崩溃（Crash）、异常中断、内存泄漏或断言失败等安全漏洞的自动化动态测试技术[20]。自 20 世纪 80 年代末由 Barton Miller 提出以来，模糊测试凭借其出色的自动化能力和对复杂软件系统的普适性，已成为学术界与工业界挖掘软件漏洞最有效的方法之一[2]。

2.2.1 模糊测试的基本流程

一个典型的模糊测试流程通常包含以下五个核心阶段：

1) **目标分析**：确定被测目标（Target）的输入接口、预期格式、依赖库及执行环境。
2) **测试用例生成**：基于特定算法生成初始输入，或从既有的种子语料库（Seed Corpus）中挑选并应用变异算法生成新的测试数据。
3) **用例执行**：将生成的测试数据输入目标程序，并触发其运行，通常结合插桩技术以获取运行时信息。
4) **状态监控**：实时捕捉程序的运行状态，识别并记录导致程序异常行为的输入样例。
5) **结果分析**：对触发异常的测试用例进行去重、精简（Minimizing），并辅助安全研究员溯源漏洞成因。

2.2.2 模糊测试的分类

根据测试过程中对目标程序内部信息的掌握程度，模糊测试主要分为黑盒、白盒与灰盒三类[21]。

黑盒模糊测试（Black-box Fuzzing）。测试者完全不了解程序的内部逻辑或源码，仅通过观测输入与输出的对应关系来发现漏洞。其优点是兼容性强，无需源码，但由于缺乏内部反馈，测试盲目性较高，难以触及深层代码逻辑。

白盒模糊测试（White-box Fuzzing）。通过符号执行（Symbolic Execution）或动态污染分析等技术，系统地探索程序的每一条路径[22]。其执行效率虽然较高，但在处理大规模复杂程序时面临严重的“路径爆炸”问题。

灰盒模糊测试（Grey-box Fuzzing）。这是目前学术界与工业界最为主流的范式。它通过轻量级的代码插桩获取程序执行过程中的代码覆盖率（Coverage）反馈，并以此引导变异策略，在测试效率与探索深度之间取得了平衡。代表性工具包括 AFL[23] 及其各类变体。

2.2.3 协议模糊测试

协议模糊测试（Protocol Fuzzing）是将模糊测试技术应用于网络协议实现的一种特定领域的测试方法。与常规的文件模糊测试不同，协议模糊测试的核心在于处理通信过程中的交互性与规范性。有效的模糊测试器必须在生成测试用例时兼顾协议的语法（Syntax）与语义（Semantics），以确保畸形数据能够通过协议解析器的基础校验，从而触达深层的业务处理逻辑。此外，针对具有复杂逻辑的协议，测试工具还需具备状态感知能力，通过模拟合规的状态变迁路径，将测试负载精准投递至特定的协议执行阶段。

根据测试用例产生方式的不同，现有的协议模糊测试工具主要可归纳为基于变异（Mutation-based）和基于生成（Generation-based）两种范式[2]。图 2-3 形象地展示了这两种范式的差异。

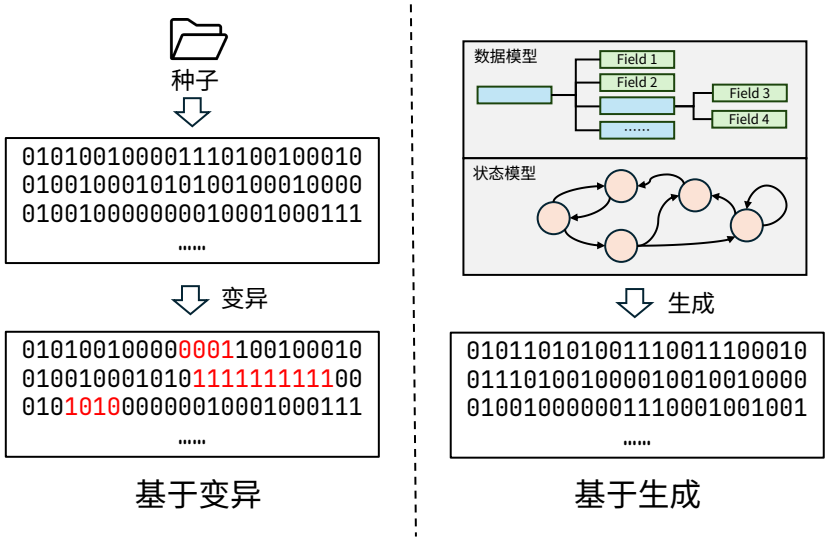


图 2-3 协议模糊测试的分类

Figure 2-3 Taxonomy of Protocol Fuzzing

基于变异的协议模糊测试通常不关注协议报文的具体结构，以已知的、合法或半合法的协议通信数据包为起点，通过应用随机或启发式的变异算子（如位翻转、字节替换、数据块插入与删除等）生成大量的测试用例。这种方法的核心优势在于其较低的部署门槛：测试人员无需深入了解协议的内部细节和规范即可开展测试[8]，尤其适用于缺乏公开文档的私有协议或黑盒测试场景。然而，网络协议往往具有严格的语法格式和校验机制，简单的盲目变异极易破坏数据包的结构完整性，导致生成的测试用例在协议解析的极早期阶段便被目标程序丢弃，难以触发深层次的代码逻辑和复杂的状态转移。在基于变异的协议模糊测试工具中，AFLNet[13]、SGFuzz[14] 和 NSFuzz[15] 是具有代表性的研究工作，它们通过引入状态感知的变异策略和网络环境

适应机制，在一定程度上提升了模糊测试的效率和效果。

基于生成的协议模糊测试则要求研究人员对协议的输入格式进行显式的建模，定义协议字段的类型、长度、取值范围以及状态机等信息，以指导测试用例的生成。模糊测试引擎依据这些预定义的规则和模板，构造出符合协议语法和部分语义规则的数据包。经典工具如 SPIKE、Sulley 及其衍生的 Boofuzz[11]，以及基于 XML 建模的 Peach[7]等，均属于此类。由于生成的测试数据包在结构上大体符合协议规范，它们能够轻易绕过前期的格式校验逻辑，从而有更高的概率深入探索目标程序的深层逻辑。尽管有效性较高，但该范式的局限性同样明显：它高度依赖于人工编写协议规范或数据模型，工具的效果也在很大程度上取决于模型的准确性和完整性。对于庞大复杂的现代网络协议或是未公开的工业控制协议，构建准确且覆盖全面的模型需要耗费巨大的专家人力成本，且工具的通用性和可扩展性较差。

2.3 Peach 模糊测试框架

Peach Fuzzer[7] 是一个开源的模糊测试框架，最初在 2004 年前后由 Michael Eddington 开发。它提出了一种基于 XML 建模的通用框架，使得研究人员能够通过编写配置文件（Pit 文件）来描述复杂的协议结构，用于生成和执行模糊测试用例。

目前在安全研究领域广泛使用的 `gitlab/protocol-fuzzer-ce`，是 Peach 社区版的一个重要开源维护分支。它保留了 Peach 的核心引擎架构，由于其跨平台性以及对工控协议、网络协议和文件格式的深度支持，至今仍是物联网和工业互联网安全研究中的主力工具之一。

2.3.1 核心架构

Peach 的架构设计遵循高度模块化的原则。如图 2-4 所示，整个框架由以下关键组件构成：

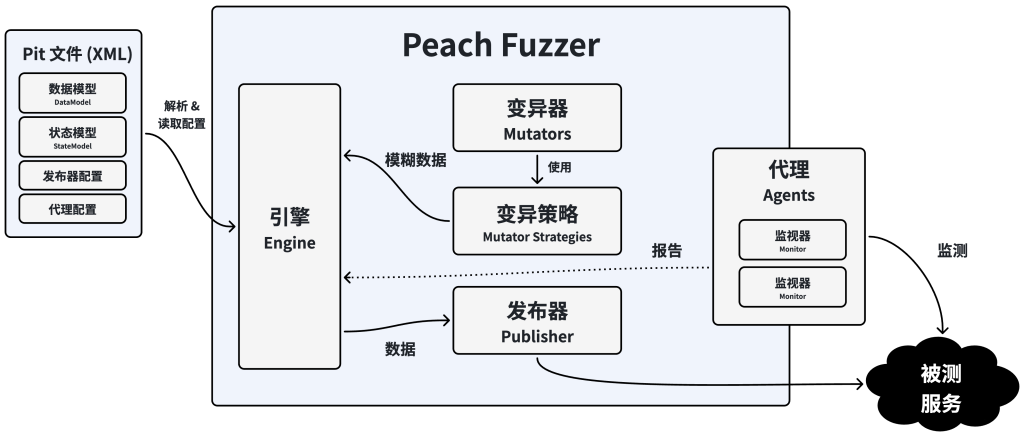


图 2-4 Peach 模糊测试框架架构图

Figure 2-4 Peach Fuzzer Architecture Diagram

1) **Pit 配置文件（Pit File）**：Pit 文件是 Peach 模糊测试框架的核心描述载体，采用 XML 格式对整个测试过程进行声明式建模。它不仅通过数据模型精确刻画输入数据的语法结构（包括字段类型、长度约束以及字段间依赖关系），还借助状态模型描述协议交互或程序执行的状态迁移逻辑，从而能够表达具有上下文语义的复杂测试流程。此外，Pit 文件还整合了发布器、监视器等组件的配置，使测试目标的连接方式与运行时监控机制得以统一描述。这种将测试知识结构化、模块化的设计，使得测试逻辑可以被复用、扩展，并在不同场景下快速迁移。
2) **引擎（Engine）**：Peach 引擎是整个模糊测试过程的调度与执行核心，负责解析 Pit 文件并将其转化为可执行的测试流程。在运行过程中，引擎调用变异策略，根据数据模型生成符合基本格式约束的测试输入，并结合状态模型驱动状态转换，从而实现对目标程序的系统化探索。同时，引擎还负责协调发布器进行输入投递，并与代理/监视器通信以获取目标程序的运行反馈（如崩溃、异常或挂起）。通过这种集中式控制机制，引擎实现了测试用例生成、执行与结果收集的闭环管理，是连接各功能模块的关键枢纽。
3) **代理与监视器（Agent and Monitor）**：代理与监视器共同构成了 Peach 的运行时观测与反馈机制，是实现自动化漏洞检测的重要基础。代理通常作为独立进程部署在目标环境中，可以是本地或远程系统，其主要职责是加载并管理多个监视器组件。不同类型的监视器提供了多样化的监控能力，例如调试器型监视器（如 WindowsDebugger、LinuxDebugger）可以捕获崩溃时的寄存器与调用栈信息，而轻量级监视器则可以通过检测进程状态判断程序是否异常终止。一旦检测到错误，监视器会立即收集相关上下文信息并通过代理回传给引擎，从而支持后续的漏洞分析与复现。
4) **发布器（Publisher）**：发布器是 Peach 中用于抽象输入输出操作的关键组件，负责将生成的测试数据传递给目标系统，并在必要时接收响应数据。通过对底层通信机制（如 TCP、UDP、文件系统等）的统一封装，发布器屏蔽了具体传输细节，使得测试逻辑能够独立于具体执行环境。例如，同一份数据模型可以在不做修改的情况下，通过替换发布器，从网络协议测试切换为文件格式测试。这种解耦设计显著提升了测试模型的通用性与可移植性，同时也便于扩展支持新的通信方式或接口类型。
5) **变异器与变异策略（Mutators and Mutation Strategies）**：变异器与变异策略是 Peach 实现输入空间探索的核心机制。变异器定义了具体的修改方式，例如针对数值字段进行边界值替换、对字符串进行插入或截断、或对结构化数据进行

重排与嵌套变换等，从而生成具有异常特征但仍满足基本格式约束的测试用例。在此基础上，变异策略则决定了不同变异器的调度方式与应用顺序，例如顺序遍历、随机选择或“确定性随机”。这使得 Peach 能够在保证输入有效性的同时，高效地覆盖更广泛的程序路径，提高发现深层漏洞的概率。

2.3.2 数据模型与状态模型

在 Peach Pit 文件中，数据模型（DataModel）通过层次化的 XML 元素对二进制或文本数据的结构进行精确描述。其基本构成包括整数（`Number`）、字符串（`String`）以及原始数据块（`Blob`）等基础元素，并支持通过组合这些元素构建复杂的数据结构。Peach 拥有在一定程度上对字段间语义关系的建模能力，例如通过 `Relation` 元素自动维护长度字段的一致性，以及借助 `Fixup` 机制（如 CRC32、MD5）在变异后动态更新校验值。这种机制保证了生成的测试用例在语法层面依然有效，从而能够穿透目标程序的浅层校验逻辑，触达更深层的执行路径。此外，Peach 还支持数据解析（Data Cracking），即在接收目标返回数据时，根据既定数据模型将原始字节流反向解析为结构化表示，为后续状态决策与数据依赖提供支持。

图 2-5 展示了一个简单的 UDP 数据包的数据模型示例，其中定义了源端口、目的端口、长度和校验和等字段，并通过 `Relation` 和 `Fixup` 元素实现了字段间的约束关系和动态修复机制。

```
<DataModel name="UdpPacket">
  <Number name="SrcPort" size="16" endian="big" />
  <Number name="DestPort" size="16" endian="big" />
  <Number name="Length" size="16" endian="big">
    <Relation type="size" of="UdpPacket" />
  </Number>
  <Number name="CheckSum" size="16" endian="big" >
    <Fixup class="UDPChecksumFixup">
      <Param name="ref" value="UdpPacket"/>
    </Fixup>
  </Number>
  <Blob name="Data" />
</DataModel>
```

图 2-5 数据模型示例

Figure 2-5 Example of DataModel

状态模型（StateModel）用于描述模糊测试过程中与目标系统的交互流程及其状态演化。该模型以状态为基本单元，通过一系列动作（Action）定义具体的交互行为，例如 `output` 用于发送构造的数据，`input` 用于接收并解析响应，而 `slurp` 则用于

从当前数据流中提取关键字段并存储为内部变量。在此基础上，状态模型通过显式的状态迁移机制（如 changeState）实现对协议语义的建模，使得后续测试行为能够依据目标反馈动态调整执行路径。例如，在典型的认证流程中，仅当先前交互满足特定条件时，系统才会进入后续功能状态。该种状态感知能力有效避免了无效或不符合上下文的测试输入，从而提升了模糊测试的效率与覆盖深度。

图 2-6 展示了一个简单的状态模型示例，其中定义了两个状态：InitialState 和 PongPongBack。在 InitialState 中，系统首先接受输入并解析为数据模型 Ping，然后根据输入内容决定是否切换到 PongPongBack 状态，最后输出相应的数据模型（Pong 或 PongPong）。

```
<StateModel name="TheStateModel" initialState="InitialState">
  <State name="InitialState">
    <Action type="accept" />
    <Action type="input">
      <DataModel ref="Ping" />
    </Action>
    <!-- Switch states only when input was PINGPING -->
    <Action type="changeState" ref="PongPongBack"
      when="state.actions[1].dataModel.find('PingPingStr') != None"
        />
    <Action type="output">
      <DataModel ref="Pong" />
    </Action>
  </State>
  <!-- This state is only reached when input was PINGPING -->
  <State name="PongPongBack">
    <Action type="output">
      <DataModel ref="PongPong" />
    </Action>
  </State>
</StateModel>
```

图 2-6　状态模型示例

Figure 2-6　Example of StateModel

2.3.3　基本流程

Peach 框架进行模糊测试时，是以迭代（Iteration）为基本单位的。图 2-7 展示了一个迭代的基本流程。

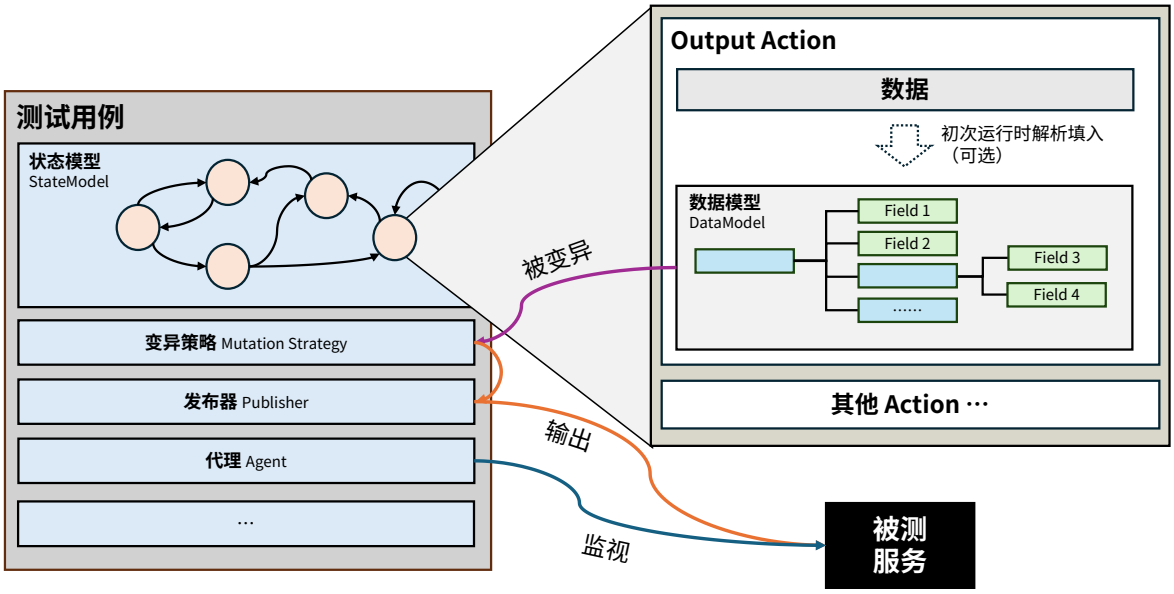


图 2-7　Peach 模糊测试流程示意图

Figure 2-7　Peach Fuzzer Workflow Diagram

首先，测试用例由状态模型（StateModel）与数据模型（DataModel）协同定义。其中，状态模型描述了测试过程中各个状态之间的转换关系，从而刻画协议交互的时序逻辑；而数据模型则以层次化方式定义了消息的内部结构，包括各字段的类型、长度及约束条件。此外，如果测试用例中定义了外部种子数据，则引擎会首先从文件中加载这些数据，并根据数据模型的定义进行解析和结构化处理，填入数据模型中对应的字段位置，作为该字段的默认值。通过这种结构化建模方式，Peach 能够生成语义上更为合理的初始测试输入，而非简单的字节流拼接。

接着，进行变异生成。变异策略（Mutation Strategy）组件根据预定义策略，调用相应的变异器，对数据模型中的字段进行扰动。该过程作用于其数据模型树上的一个或多个字段，生成多样化的测试输入。变异操作通常不会破坏数据的基本结构（如长度字段与实际数据长度的一致性），使生成的数据在保持结构合法性的同时引入潜在异常。

最后，发布器（Publisher）负责将生成的测试用例发送至被测服务，而代理（Agent）则用于监测目标环境中的被测服务。它持续观察被测服务的运行状态，包括崩溃、异常退出或资源异常等行为。一旦检测到异常，系统将记录对应的测试输入及执行上下文，以便后续分析与漏洞复现。

2.3.4　优势与局限性

Peach 在协议模糊测试领域展现出一些优势。首先，其基于数据模型的语义感知能力使其能够生成符合协议基本结构约束（如长度字段、校验和等）的高质量测试用

例，从而显著提高触达目标程序深层逻辑路径的概率。其次，Peach 采用高度模块化的插件式架构，支持用户灵活扩展和定制变异器、发布器与监视器等核心组件，使其能够适配多样化的测试场景。此外，Peach 提供了完善的状态机建模机制，能够有效支持具有复杂交互流程的有状态协议测试，例如涉及握手、认证及多轮通信的网络协议，这使其在协议层模糊测试中具有较强的表达能力和适用性。

然而，Peach 也存在若干不可忽视的局限性。首先，其对高质量 Pit 文件的依赖导致建模成本较高，用户通常需要深入理解相关协议规范（如 RFC 文档）或借助大量逆向分析工作，这在一定程度上提高了使用门槛。其次，由于 Peach 构建于.NET/Mono 平台之上，并依赖 XML 进行数据与状态描述，其执行效率相对较低，在每秒执行次数（Exec/s）方面明显逊于 AFL、libFuzzer 等轻量级模糊测试工具。最后，Peach 缺乏基于代码覆盖率的反馈引导机制，本质上属于黑盒模糊测试方法，无法像基于覆盖率反馈的工具那样通过进化策略持续探索新的执行路径，从而在复杂程序中的路径覆盖能力受到一定限制。

2.4 大模型智能体

随着大语言模型（Large Language Models, LLM）的能力不断提升，其应用形态正从单轮文本生成逐步演进为能够执行复杂任务的智能系统。在这一背景下，大模型智能体（LLM Agent）作为一种新兴范式被提出，用以描述以大语言模型为核心，通过与外部环境交互完成多步骤任务的自主系统[24-25]。相比传统仅基于输入输出映射的语言模型，大模型智能体通过引入推理过程建模与工具调用机制，使模型具备一定程度的决策与执行能力，从而能够处理更加复杂和开放的任务场景。

2.4.1 基本概念与工作机制

从系统结构上看，大模型智能体通常运行在一个“感知—推理—行动”的闭环之中。模型首先接收来自外部环境的输入，例如自然语言指令、程序执行结果或结构化数据；随后基于当前上下文进行推理，生成中间思考过程及下一步操作；最后通过调用外部工具或接口执行具体动作，并将执行结果反馈给模型以更新其内部状态。这一过程往往以迭代方式进行，使得模型能够在多轮交互中逐步逼近目标。

这种机制的关键在于将推理过程显式化，使模型不仅输出最终结果，还能够生成中间决策步骤，从而提升复杂任务的可解释性与稳定性。同时，通过引入外部工具（如代码执行器、搜索引擎或数据库接口），大模型智能体能够突破纯文本生成的能力边界，将语言理解与实际操作相结合。

2.4.2 挑战与发展趋势

尽管大模型智能体展现出广泛应用潜力，但其发展仍面临若干挑战。首先，由于大语言模型具有一定的生成不确定性，在多步推理过程中可能出现错误累积，从而影响最终结果的可靠性。其次，工具调用与环境交互通常伴随着额外的计算开销与系统复杂性，这在一定程度上制约了其实际部署效率。此外，大模型智能体的行为在很大程度上依赖于提示设计与控制策略，其泛化能力在不同任务之间仍有待进一步验证。

提升智能体的推理稳定性、降低执行成本，以及构建更加通用且可解释的决策机制，是当前研究的热点方向。同时，如何将大模型智能体与具体应用领域深度融合，也是推动其落地的重要方向。

3、 协议语义感知变异增强方法

为了解决传统协议模糊测试工具对协议语义理解不足的问题，本文设计了一种大语言模型辅助的协议语义感知变异增强方法。该方法设计主要包括两个部分——**大模型智能体驱动的测试组件生成流水线**，以及适配生成组件的**两阶段变异策略**。

3.1 方法概述

图 3-1 展示了该方法的整体概览。该方法利用大语言模型的代码生成与语义理解能力，通过构建基于大模型智能体（LLM Agent）的自动化流水线，实现从协议规范解析到测试组件生成的端到端自动化流程。借助该流水线，可以针对新协议自动生成包括数据模型、变异器以及修复器，快速构建出具有协议语义感知能力的测试组件，显著降低了协议建模的人工成本。

进一步地，该方法还对现有的成熟模糊测试框架 Peach 进行了拓展与适配，设计了 LLMPeach 测试框架。该框架为大模型生成的测试组件提供了接口和检验器，并设计了两阶段变异策略，以充分利用生成组件的能力，同时不放弃传统变异器的带来的随机性，从而兼顾测试用例的多样性与语义合理性。

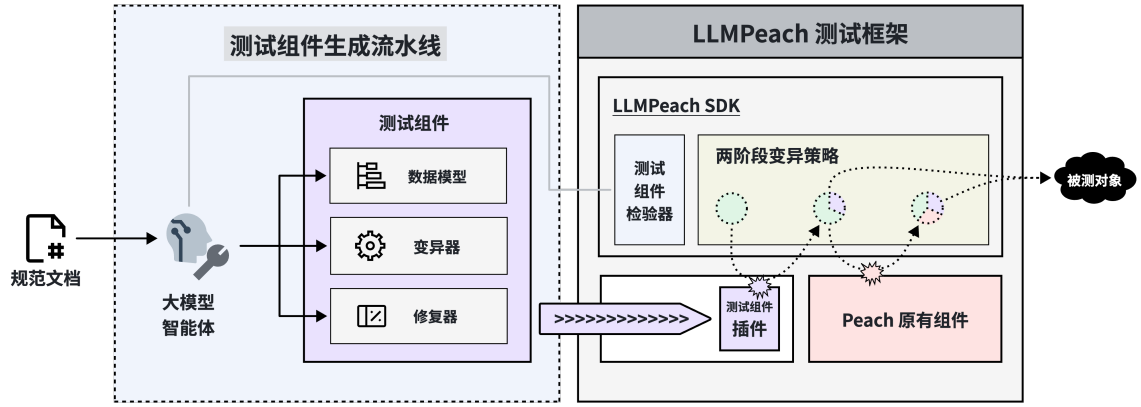


图 3-1　协议语义感知变异增强方法概览

Figure 3-1　Overview of the Protocol Semantic-Aware Mutation Enhancement Method

3.2 大模型智能体驱动的测试组件生成流水线

图 3-2 展示了该流水线的整体设计。首先，系统通过检索增强生成（RAG）技术构建知识库，并引导大语言模型提取协议的报文类型。随后，针对每种报文类型，系统进一步引导模型进行字段建模，生成包含字段含义、取值约束及逻辑关联的结构化数据模型。最后，大语言模型基于数据模型构造出语义相关的变异器（Mutator）；结合协议的约束规则，构建自适应修复器（Fixer）以纠正因变异引入的语义违规行为。

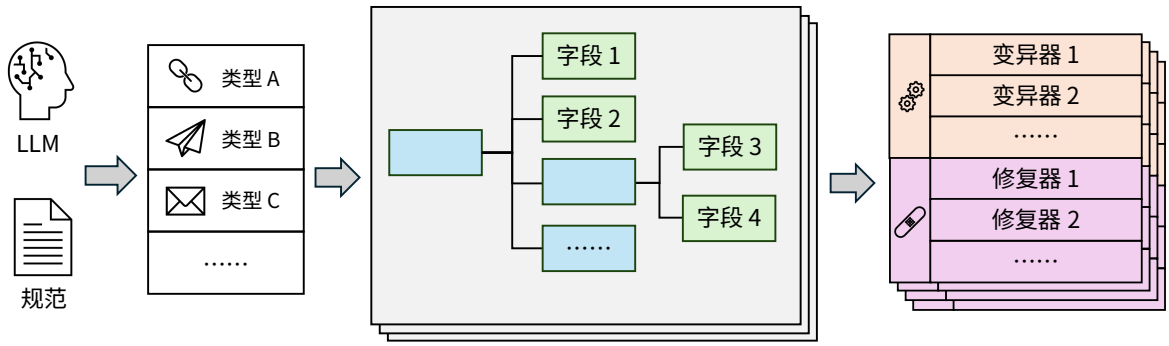


图 3-2　大模型智能体驱动的测试组件生成流水线设计

Figure 3-2　Design of the LLM Agent-Driven Test Component Generation Pipeline

为了高效地理解协议规范，系统中引入了检索增强生成[26]（Retrieval-Augmented Generation, RAG）技术。RAG 技术允许大语言模型在生成过程中实时访问外部知识库——即目标协议的规范文档。通过将长篇幅的协议文档向量化并存储于检索器中，系统能够根据当前的解析需求，在知识库中进行检索，提取出协议规范文档中的相关段落，并将其作为上下文输入给模型。这种方式不仅解决了大语言模型在处理超长文档时的上下文窗口限制问题，还能够降低大模型产生“幻觉”的概率，确保生成的数据结构描述具有权威的文档支撑。

3.2.1　报文类型提取

“报文类型提取” 是整个生成任务的第一步，其目标是识别出该协议中所有的报文类别。图 3-3 展示了用于报文类型提取的提示词设计。其旨在驱动大语言模型智能体调用检索工具（如 RFC_Search），在海量的文档文本中定位涉及报文格式定义的章节，并根据这些章节的内容，忽略冗余的叙述性文字，仅抽取出协议报文的类型列表，并以特定的格式输出。

Task: For <protocol> protocol, list ALL the packet types according to the RFC document.
Use the “RFC_Search” tool to look up the relevant sections in the RFC.
Return the list as a comma-separated string. ONLY output the types, nothing else.
[Example]

Tool: RFC_Search

图 3-3　报文类型提取的提示词设计[1]

Figure 3-3　Prompt Design for Packet Type Extraction

1　因大多数大语言模型的训练语料以英文为主，该提示词以英文设计，下同。

提取出的报文类型列表将作为元数据存储于系统的全局状态中，为后续针对每种具体报文进行建模提供明确的目标。

3.2.2 数据模型生成、检验与修复

在获取了协议的报文类型清单后，流水线进入第二阶段——针对每种特定报文构建精细化的 Peach 数据模型。此步骤是整个生成流水线的关键，因为数据模型的质量将直接影响后续变异器和修复器的效果。大语言模型需要深入理解每个字段的基本含义（位宽、偏移、类型）以及语义关联（校验和、长度依赖、状态标志），并将这些信息以 Peach 数据模型的 XML 格式进行建模。

为了确保生成的数据模型的准确性与实用性，设计方案并未采用一次性的直接生成策略，而是构建了一个“生成—验证—修复”的闭环流水线，以应对大语言模型在生成结构化数据时可能出现的格式错误、语义误解或遗漏问题。

图 3-4 展示了数据模型生成与检验的流程设计。大语言模型首先进行数据模型的生成，随后通过检验脚本对生成的数据模型进行验证。若验证失败，系统引导模型修复错误，并重新生成数据模型。该流程将持续迭代，直到生成的模型通过所有检验项为止。

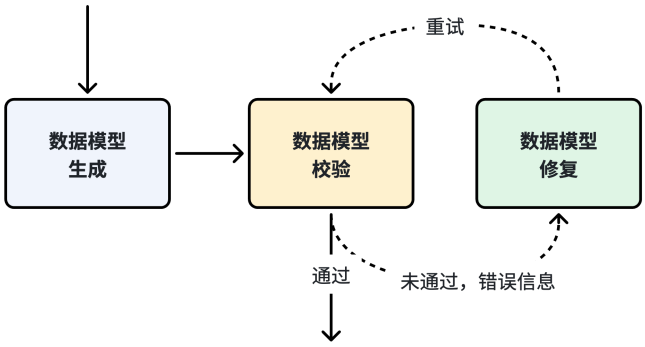


图 3-4　数据模型生成与检验流程

Figure 3-4　Data Model Generation and Validation Process

数据模型生成。如图 3-5 所示，这一阶段以先前提取的报文类型列表为输入，并结合协议规范文档中对报文格式与语义约束的描述，要求大模型对各类报文的结构进行细粒度建模。在提示词中，明确规定了模型需要遵循的建模原则，包括如何利用 XML 元素刻画字段类型、长度、取值范围以及字段间的依赖关系（如长度字段与数据字段之间的关联）。同时，通过补充示例与操作提示（如 Relation、Optional 和 Block 等关键机制的使用方式），这些操作提示促使模型能够主动运用这些机制，进一步约束生成结果的表达形式与完整性。此外，该提示词还包含了从 Peach 的说明文档中提

取的数据模型中相关数据类型的说明性文本，以帮助模型更准确地理解 Peach 数据模型的语义要求，从而生成符合规范的数据模型。

Task: Using the packet types we just identified (<packet_types>), generate a Peach Pit file defining the precise structure of each packet for <proto>.
Model the structure as detail as possible.
[Example]
Hint:
1. Read the usage of each XML element in “peach.txt” to understand how to define the structure for each packet type.
2. *Other Hints...*

Tool: RFC_Search, Write_File, Read_File

图 3-5　数据模型生成的提示词设计

Figure 3-5　Prompt Design for Data Model Generation

数据模型检验。为了尽可能提升生成的数据模型的质量，对数据模型进行检验是必不可少的环节。大模型生成的数据模型中，通常可能出现以下几类问题：1）数据类型错误，如使用了 Peach 不支持的数据类型；2）字段定义错误，如字段的位宽、偏移或取值范围与协议规范不符；3）依赖关系错误，如长度字段未正确关联到数据字段，或校验字段未正确关联到被校验的数据字段等。4）结构错误，如缺失必要的字段或包含冗余字段等。5）过度简化，如未能充分利用 Peach 的机制来表达协议的复杂语义关系，导致生成的数据模型过于简化，无法准确反映协议的实际结构和约束。

针对这些潜在问题，此阶段引入了数据模型检验器，以自动化地检测生成的数据模型是否存在上述问题。检验器会将预先准备的大量该协议的合法数据包作为测试输入，利用生成的数据模型进行解析，然后不经任何修改地将解析结果重新输出。通过比较输入与输出数据包的结构和内容是否一致，可以揭示生成的数据模型在字段定义、长度约束、依赖关系等方面是否存在错误或不完整之处。若检验失败，系统会将具体的错误信息连同解析日志一同输出。

数据模型修复。当检验阶段检测到生成的数据模型存在问题时，系统会进一步引导大语言模型对数据模型进行自动修复。图 3-6 展示了数据模型修复的提示词设计。该提示词要求模型首先分析检验日志，定位数据模型中的具体错误（如字段解析失败、长度不匹配等），并输出清晰的错误解释。随后，模型需要根据错误信息对原有的数据模型进行针对性的修改，以修正这些问题。需要强调的是，在修复过程中，大模型被禁止简化数据模型的结构，避免其以过度简化的方式来规避问题，而是需要在

保持数据模型完整性的前提下进行修复，以确保最终生成的数据模型能够准确反映协议的实际结构和约束。

Task: We ran a verification test against the generated datamodel, which used the Peach Pit file to parse packets and check if the fields were correctly extracted. The test failed, indicating there are issues with the datamodel.
Here is the test output: *[Validation Output]*
You need to:

1. Read EACH test log file in <test_log_directory> to identify the specific issues with the datamodel. Look for parsing errors, mismatched fields, or any indications of what part of the datamodel is incorrect.
2. For each identified issue, output a clear explanation of what the problem is and which part of the datamodel it relates to.
3. Modify the Peach Pit file to fix the identified issues.

CRITICAL: Simplifying the DataModel is NOT allowed.

Tool: Read_File, Write_File

图 3-6　数据模型修复的提示词设计

Figure 3-6　Prompt Design for Data Model Repair

3.2.3　变异器的生成、检验与修复

在本阶段中，将进行变异器（Mutator）的构建。变异器决定了模糊测试工具如何在遵循协议基本语义的前提下，探索目标程序的边界条件与逻辑漏洞，被视为模糊测试的核心组件之一。

为了实现更深层、更具语义感知的测试，本方法不再依赖 Peach 框架内置的通用变异算子，而是利用大语言模型针对协议的每一个字段生成定制化的 C# 变异逻辑。这些定制变异器能够理解字段的业务逻辑，例如生成常见值、边界值等。

如图 3-7 所示，与数据模型的生成流程不同，变异器在生成时采取了并行化的设计思路，即针对每个字段的变异器生成任务可以同时进行，以提升整体效率。另一方面，这样处理也能够减少每个变异器生成时的上下文复杂度，使得大语言模型能够更专注于单一字段的语义特征，从而生成更精准的变异逻辑。同样，为了确保生成的代码能够正常编译并符合 Peach 的接口规范，该过程依然遵循“生成—验证—修复”的循环逻辑。

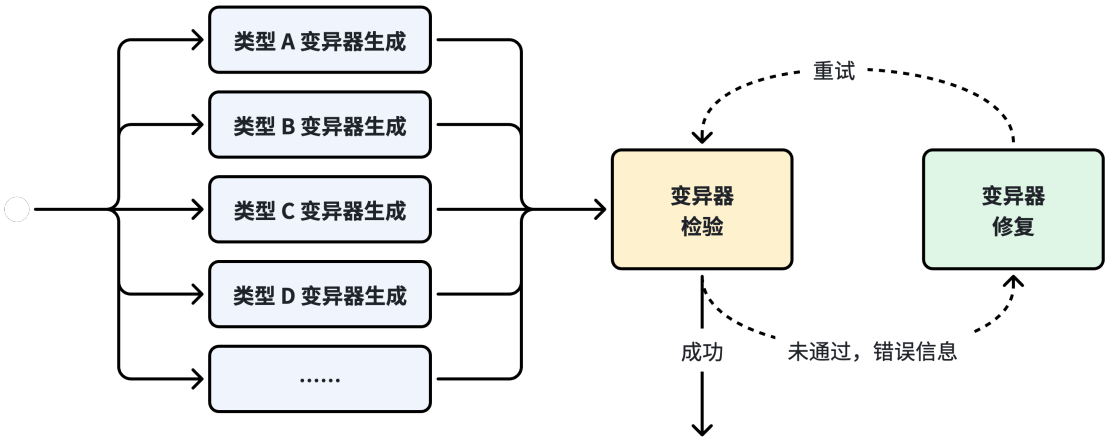


图 3-7　变异器生成、检验与修复流程

Figure 3-7　Mutator Generation, Validation and Repair Process

变异器生成。如图 3-8 所示，在此阶段，对于每种协议包类型，大语言模型需要根据规范文档和先前生成的数据模型，先列出该消息包类型中包含的全部字段。对于每个字段，大模型需要对其分类处理——对于取值固定的字段，不生成任何变异逻辑；而对于可变字段，则根据其结构特征（如是否可选、是否可重复）生成对应的结构性操作类（例如 Add、Remove、Repeat）。

对于可变字段的设计是变异器生成的核心。提示词明确规定需要围绕字段的局部语义属性构建多维度的变异策略，具体涵盖八类语义范畴：A) 规范形式（Canonical form），用于生成符合标准语义的推荐取值；B) 边界值（Boundaries），用于探索字段取值空间的上下界及临界点；C) 等价类替代（Equivalence-class alternatives），通过替换为语义上有效但结构不同的值以触发潜在分支；D) 合法位域/枚举/范围（Allowed bitfield/enum/range），适用于具有明确取值集合的字段；E) 编码形态变体（Encoding-shape variant），例如不同编码方式或长度表示形式的切换；F) 填充与对齐（Padding/alignment），用于测试实现中对内存对齐或填充字节的处理逻辑；G) 前缀/后缀扰动（Prefix/Suffix），通过在字段前后添加特定模式以探测解析器的鲁棒性；H) 随机合法混合（Random valid mix），用于生成组合型输入以提升测试多样性。

总的来说，变异器生成阶段以协议字段为核心，将其按可变性与语义特征进行系统化拆解，并围绕多维语义范畴设计针对性的变异策略。通过结合结构性操作与语义驱动的取值生成方法，不仅能够覆盖常规与边界输入，还能探索多样化且具有潜在触发效果的测试数据，从而提升协议实现的测试深度与缺陷发现能力。

Task: List ALL fields for the <protocol> <packet_type> packet.
For EACH field <field_name> in the <protocol> <packet_type> packet:

1. **Fixed value?** If the field is fixed, do not generate any mutator functions.
2. **Otherwise (the field is mutable):**

   (a) If the field is optional, implement:

   ```
   class <PROTO><PacketType>Add<FiledName>
   class <PROTO><PacketType>Remove<FiledName>
   ```

   (b) If the field may appear multiple times, also implement:

   ```
   class <PROTO><PacketType>Repeat<FiledName>
   ```

   (c) **Mutate.** Design semantic-aware mutators for this field by covering the following field-local semantic categories:

   A. Canonical form　　B. Boundaries
   C. Equivalence-class alternatives　　D. Allowed bitfield/enum/range
   E. Encoding-shape variant　　F. Padding/alignment
   G. prefix/suffix　　H. Random valid mix

   Add randomized perturbations mixing shallow and deep changes to preserve long-term diversity and avoid collapse into a single pattern.

   ```
   class <PROTO><PacketType>Mutate<FiledName>
   ```

Write in C# 5.0. Inherit from <class LLMMutator> in the LLMPeach SDK.
[Example]

Tool: RFC_Search, Write_File, Read_File, Build_DotNet_DLL, Search_Class

图 3-8　变异器生成的提示词设计

Figure 3-8　Prompt Design for Mutator Generation

在变异器的生成过程中，大模型智能体可以利用多种辅助工具（Tool）提升生成质量。其中，`RFC_Search` 用于检索协议规范中的字段定义与约束信息，为变异逻辑提供权威的语义依据；`Read_File` 与 `Write_File` 分别用于读取模板代码与输出生成结果；`Build_DotNet_DLL` 用于对生成的 C# 代码进行编译验证。特别地，`Search_Class` 工具可从 LLMPeach SDK 已编译的库文件中检索指定类，并展示其方法接口与继承结构，从而帮助模型准确理解库中相关类（如 `LLMMutator`）的使用方式，减少接口误用带来的生成错误。通过上述工具的协同配合，可以有效提升变异器生成结果的正确性与工程可用性。

变异器检验。变异器的检验与数据模型的检验有所不同。检验脚本会在合法的测试输入上运行生成的变异器，并检查其变异的结果。每个变异器都将被测试数百次，

以减少随机性的影响。对于每一个被测试的变异器，如果其变异途中发生了错误，或在 100 轮测试中未能成功生成任何一个合法的变异结果，则认为该变异器存在问题，需要进入修复流程。

变异器修复。如图 3-9 所示，这一阶段以变异器验证测试的错误输出为输入，指导大语言模型对存在缺陷的变异器进行定位与修复。提示词要求模型根据报错信息分析异常原因，读取对应的 C# 源文件，并针对关键逻辑中可能存在的空值访问、越界操作或不合理处理进行修正。在完成修改后，通过 `Write_File` 更新代码，并利用 `Build_DotNet_DLL` 进行重新编译验证，从而确保修复后的变异器在语法与运行层面不会触发异常。

Task: We ran a verification test against the generated mutators. The test failed with an ERROR for the mutator <mutator_name>.
Here is the test error output for this mutator:
[Validation Output]
The test logs indicate there are issues with the generated C# code for <mutator_name>.
You need to:

1. Find the C# file containing the <mutator_name> class.
2. Analyze the traceback and error message to understand the logic flaw or runtime exception.
3. Use the “Read_File” tool to read the corresponding mutator file.
4. Fix the bug in the C# code. Make sure to handle potential nulls, index out of bounds, etc., that might occur during ‘PerformMutation’.
5. Use the “Write_File” tool to update the file with the fix.
6. Use the “Build_DotNet_DLL” tool to recompile the mutators and ensure there are no syntax errors.

Be thorough and ensure the C# code will successfully compile.

Tool: Read_File, Write_File, Build_DotNet_DLL

图 3-9　变异器修复的提示词设计

Figure 3-9　Prompt Design for Mutator Repair

3.2.4　修复器的生成、检验与修复

在此阶段，将完成修复器（Fixer）的构建。修复器可以修复在变异过程中可能引入的语义违规行为，以确保生成的测试用例在保持多样性的同时，仍然能够满足协议的基本语义约束，从而提高测试效率和有效性。

图 3-10 展示了修复器的生成、检验与修复流程设计。与变异器类似，修复器的

生成也遵循“生成—验证—修复”的循环逻辑，以提高其功能的正确性与实用性。此流程主要分为约束提取、修复器生成、约束修复测试生成、修复器检验与修复器修复五个模块。首先，大模型会提取该协议规范中的约束，随后，针对这些约束，并行地生成修复器代码和约束修复测试用例。接着，检验模块会运行修复器并检查其是否能够成功修复变异引入的语义违规行为。若检验失败，系统会引导模型分析失败原因并对修复器进行针对性的修正，直到生成的修复器能够通过所有检验项为止。

值得注意的是，虽然针对的是相同的约束，修复器的生成和约束修复测试的生成是分别进行的。在进行约束修复测试的生成时，模型没有生成修复器的记忆，提示词也不允许模型查看对应约束的修复器代码，以确保测试用例的独立性和有效性。

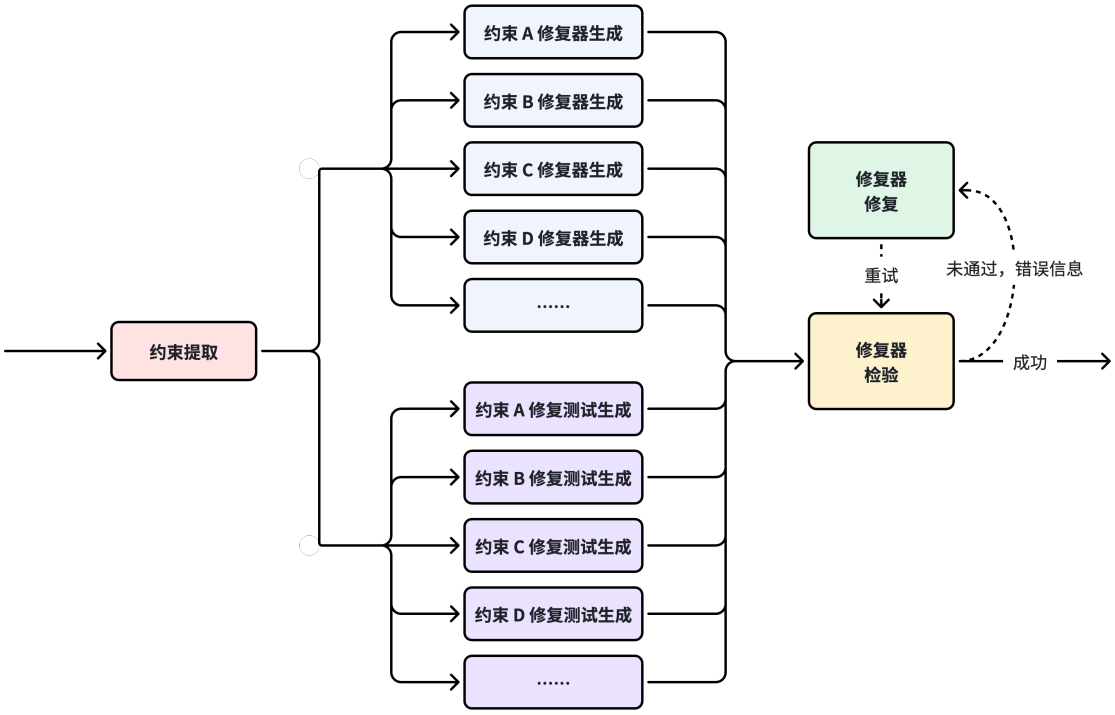


图 3-10　修复器生成、检验与修复流程

Figure 3-10　Fixer Generation, Validation and Repair Process

约束提取。如图 3-11 所示，这一阶段以协议规范文档为输入，要求大语言模型从 RFC 中系统性地提取所有与客户端到服务器请求报文相关的格式约束。提示词引导模型重点关注规范中以强制性语言（如 MUST、MUST NOT、SHALL 等）描述的规则，并将其整理输出，为后续修复器的生成与验证做好准备。

Task: Extract all constraints related to request (client to server) message format from the <proto> RFC. For example, in MQTT there are the following constraints:

- [MQTT-2.2.1-2] A PUBLISH packet MUST NOT contain a Packet Identifier if its QoS value is set to 0.
- [MQTT-3.1.2-11] If the Will Flag is set to 0, then the Will QoS MUST be set to 0 (0x00).
- ...

Tool: RFC_Search

图 3-11　约束提取的提示词设计

Figure 3-11　Prompt Design for Constraint Extraction

修复器生成。如图 3-12 所示，该阶段围绕已提取的协议约束，要求大语言模型为每条约束构建对应的修复逻辑，从而对不符合规范的报文进行就地调整。

Task: For <protocol>, write fixer functions for EACH constraint below.

Write in C# 5.0. File: <fixer_file_name>.cs

[Example]

The input is a single packet. Fix in place.

HINTS

- Change a field value: `obj.SetValue(new Variant(...));`
- Remove a field: `obj.parent.Remove(obj);`
- Make 'Optional' field exist: `obj.SetValue(new Variant(...))` on its child field(s) with valid value(s).

Constraints for this task: <constraint_list>

You must ensure there are NO syntax errors and the code compiles successfully.

Tool: Read_File, Write_File, Build_DotNet_DLL, Search_Class

图 3-12　修复器生成的提示词设计

Figure 3-12　Prompt Design for Fixer Generation

约束修复测试生成。如图 3-13，该提示词用于指导大语言模型基于协议约束自动生成对应的测试用例，以验证每个修复器（Fixer）的正确性。其核心目标是：针

对输入的每一条约束，首先构造一份违反对应约束的报文数据；接着调用待测试的 Fixer 函数对报文进行修复；最后通过断言验证修复结果是否恢复到满足约束的状态。

Task: For <protocol>, write NUnit test functions for validating EACH fixer constraint below.
[Constraints]
Write in C# 5.0.
[Example]
Test Workflow:

1. Construct packet data that violates the constraint.
2. Invoke the corresponding Fixer function.
3. Assert that the Fixer restores the constraint.

IMPORTANT:

- Ensure NO syntax errors and successful compilation.
- Treat Fixers as a black box; do NOT inspect their implementation.

Tool: Read_File, Write_File, Build_DotNet_DLL

图 3-13　约束修复测试生成的提示词设计

Figure 3-13　Prompt Design for Constraint Fix Test Generation

修复器检验。修复器的检验主要通过运行前面生成的约束修复测试来完成。每个修复器都将对应一个或多个测试用例，这些测试用例会构造出违反特定约束的输入数据，并调用修复器进行修正。检验脚本会检查修复后的结果是否满足原有的约束条件，从而验证修复器的功能正确性。如果某个修复器未能通过其对应的测试用例，则认为该修复器存在缺陷，需要进入修复流程。

修复器修复。当修复器检验失败时，系统会引导大语言模型对存在缺陷的修复器进行定位与修正。如图 3-14 所示，这个过程与变异器的修复类似。大模型需要分析测试失败的具体原因，读取对应的 C# 源文件，并针对关键逻辑中可能存在的问题进行修正。在完成修改后，更新代码，并重新编译验证，直到修复器能够通过所有相关测试用例为止。

Task: We ran a verification test against the generated fixers. The test failed with an ERROR for the fixer <fixer_name>.
Here is the test error output for this fixer:
[Validation Output]
The test logs indicate there are issues with the generated C# code for <fixer_name>.
[Repair Steps]

Tool: Read_File, Write_File, Build_DotNet_DLL

图 3-14　修复器修复的提示词设计

Figure 3-14　Prompt Design for Fixer Repair

3.3　两阶段变异策略

为了充分发挥大模型生成的变异器和修复器的能力，本方法设计了一套两阶段的变异策略。图 3-15 展示了该策略的基本流程。

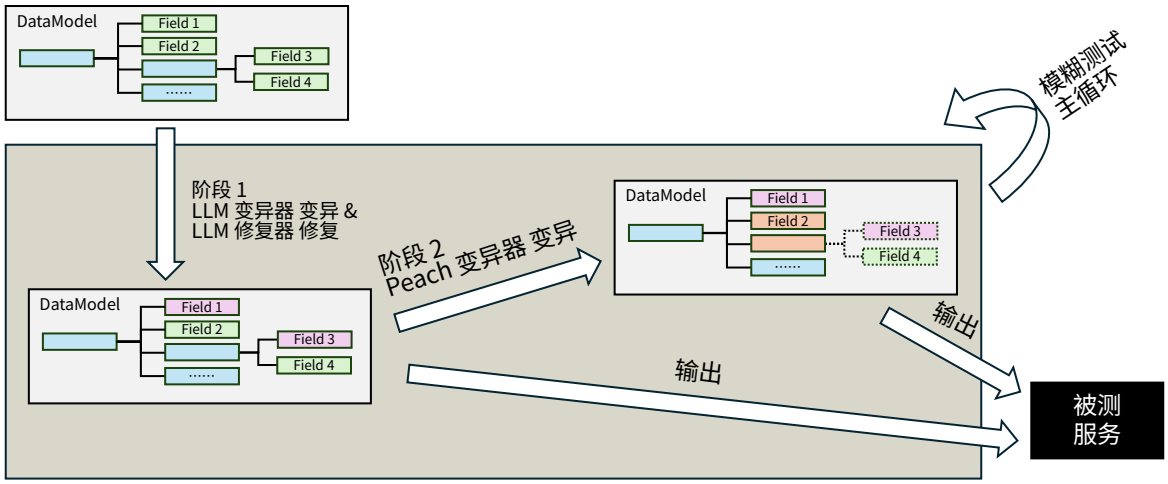


图 3-15　两阶段变异策略设计

Figure 3-15　Design of the Two-Stage Mutation Strategy

图 3-16 展示了两阶段变异策略算法。该策略将变异过程划分为两个阶段，以兼顾输入语义有效性与测试扰动强度。第一阶段采用大语言模型（LLM）生成的变异器，作用于具备语义结构的字段，对输入进行协议语义相关的变异，在扩展输入空间的同时尽可能维持协议约束与上下文一致性。此外，引入修复器对变异结果进行约束修正，以确保生成的数据满足基本语法与语义合法性，从而提高有效测试样本的比例。

第二阶段使用传统变异器（如边界值修改、随机比特翻转、枚举扰动等）对所有候选字段进行进一步变异。该阶段不再强调语义保持，而是通过更具破坏性的操作引

入异常输入，以触发潜在的边界错误、解析异常或状态机缺陷，从而增强测试的探索能力和覆盖范围。

在每个阶段中，针对单个元素的变异次数并非固定，而是从区间 $[1, k]$ 中通过加权随机分布进行采样，其中较小的变异次数具有更高概率。这种设计避免了过度变异导致的语义完全破坏，同时允许一定概率的多次连续变异，以探索更远离初始输入的状态空间，在保持数据包合法和扩大探索范围之间取得平衡。

Algorithm 1 两阶段变异策略

Require: 动作 a，变异集合 M，最大变异次数 k，LLM 变异器集合 $\mathscr{M}_{LLM}$
Ensure: 变异后的数据模型 D

$D \leftarrow a.outputData$
$S_1 \leftarrow \{e \in M \mid e.Mutators \cap \mathscr{M}_{LLM} \neq \varnothing\}$
for all $e \in S_1$ **do** ▷ 阶段一：LLM 变异
  $n \sim \mathscr{W}(1, k)$
  for $i = 1$ to n **do**
    $m \leftarrow RandPick(e.Mutators)$ ▷ 随机选择一个 LLM 变异器
  end for
end for
$D.fixup()$ ▷ 应用修复器修正语义违规
$a.Output(D)$ ▷ 输出阶段一产物
for all $e \in M$ **do** ▷ 阶段二：非 LLM 变异
  $n \sim \mathscr{W}(1, k)$
  for $i = 1$ to n **do**
    $m \leftarrow RandPick(e.Mutators \setminus \mathscr{M}_{LLM})$ ▷ 随机选择一个非 LLM 变异器
  end for
end for
$a.Output(D)$ ▷ 输出阶段二产物

图 3-16　两阶段变异策略算法

Figure 3-16　Algorithm of the Two-Stage Mutation Strategy

3.4　LLMPeach 的实现

基于上述设计，本研究实现了一个名为 LLMPeach 的协议模糊测试框架。LLMPeach 将大语言模型生成的变异器和修复器集成到 Peach 模糊测试框架中，并采用两阶段变异策略进行测试输入的生成。

3.4.1　基本架构

图 3-17 展示了 LLMPeach 的系统架构设计。在架构上，LLMPeach 由 LLMPeach SDK 和插件（Plugin）构成。LLMPeach SDK 由原有的 Peach 组件扩展而来，新增了一些数据类型，大模型变异器和修复器的接口定义以及相关的校验程序，并实现了两阶段变异策略。得益于 Peach 框架的高度模块话设计和可扩展性，本研究仅在最低限度上修改了原有组件的代码，并将此方法引入的增强与扩展作为独立模块进行集成，以最大程度地保持与原有系统的兼容性和稳定性。

插件部份则用来放置前文流水线生成的测试组件库，以供 LLMPeach SDK 动态加载。通过这种方式，LLMPeach 能够无缝地将大语言模型生成的变异器和修复器集成到 Peach 的模糊测试流程中，而无需重新编译整个系统，保持了系统的灵活性和可扩展性。

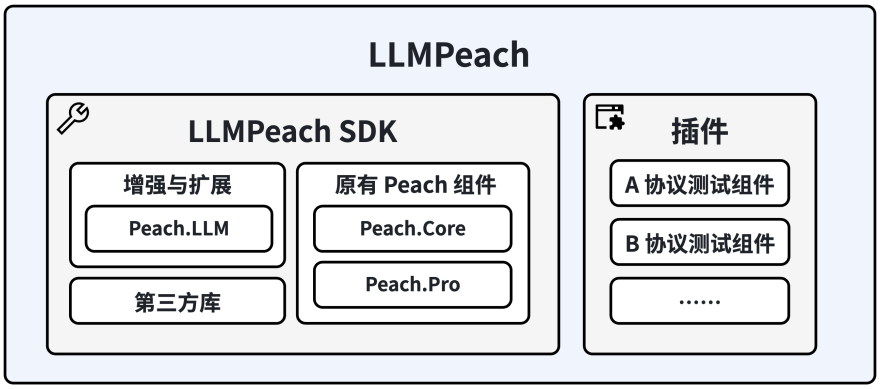


图 3-17　LLMPeach 系统架构设计

Figure 3-17　Architecture Design of LLMPeach

3.4.2　Peach 框架的增强与扩展

为了支持大语言模型生成的变异器和修复器，LLMPeach 对 Peach 框架进行了多方面的增强与扩展。

Optional 数据类型。首先，在 Peach 的数据模型层面，LLMPeach 引入了 Optioal 数据类型，以弥补 Peach 原有数据模型在表达协议中可选字段时的不足。具体而言，Optional 作为一种条件数据容器，用于根据给定条件动态决定其内部字段是否参与解析与生成。

该类型通过两个核心属性实现语义控制：一是 src，用于指定参考字段路径；二是 expression，用于定义基于参考字段取值的逻辑表达式。在运行时，系统首先解析 src 指向的字段，并将其值绑定为表达式中的变量 value，随后对 expression 进行求值。当表达式结果为真时，Optional 内部的子元素将被正常解析与参与数据生成；否则，该结构将被跳过，从而实现对协议中条件字段的精确建模。

在数据解析（cracking）阶段，Optional 类型会在处理前动态评估条件表达式，仅在条件满足时才对子元素进行递归解析；否则直接跳过对应数据片段，从而避免无效解析带来的错误传播。在数据生成阶段，Optional 同样依据条件结果决定是否输出对应字段，若条件不满足，则生成空值或零长度数据，以保持整体数据结构的一致性。

图 3-18 展示了一个 Optional 数据类型的使用示例。在该示例中，will_optional 作为一个 Optional 元素，其解析与生成完全依赖于 connect_flags 字段的特定位（bit 2）

的值。当该位被置为 1 时，will_optional 内部的 will_props、will_topic 和 will_payload 等字段将被正常处理；反之，则整个 will_optional 结构将被跳过，从而实现了对 MQTT 协议中遗嘱消息相关字段的条件建模。

```
<DataModel name="mqtt_connect_payload_t">
    <Block name="client_id" ref="MQTT_String"/>

    <!-- Will Flag (bit 2) -->
    <Optional name="will_optional"
       src="variable_header.connect_flags" expression="(value &amp;
       0x04) != 0">
      <Block name="will_props" ref="mqtt_properties_t"/>
      <Block name="will_topic" ref="MQTT_String"/>
      <Block name="will_payload" ref="MQTT_BinaryData"/>
    </Optional>

    <!-- Other fields... -->
</DataModel>
```

图 3-18　Optional 数据类型示例

Figure 3-18　Example of Optional DataElement

大模型变异器与修复器基类。为了规范大语言模型生成的变异器和修复器的接口与行为，LLMPeach 在 SDK 中定义了 LLMMutator 和 LLMFixer 两个抽象基类。这些基类继承自 Peach 框架中相应的 Mutator 和 Fixer 基类，并在此基础上增加了针对大模型生成组件的特定接口要求和功能约束。

测试组件检验器。为了确保大语言模型生成的变异器和修复器能够正确编译并符合 Peach 的接口规范，LLMPeach 实现了一套自动化的测试组件检验器。该检验器实现了前文设计的变异器和修复器的检验流程，并将其提供给大模型智能体驱动的测试组件生成流水线使用。该检验器，能够在生成阶段后运行验证测试，并根据测试结果提供详细的错误报告，反馈给生成流水线，从而引导模型进行针对性的修正。

两阶段变异策略实现。LLMPeach 模糊测试框架按照 Peach 的策略规范实现了前文设计的两阶段变异策略。

4、 实验评估与分析

针对协议语义感知变异增强方法及其在 Peach 框架上的实现（LLMPeach），本文设计了三个研究问题（RQs）来评估其性能和有效性：

- **RQ1**：相较于原有 Peach 框架，LLMPeach 能否在被测协议实现上达到更高的分支覆盖？
- **RQ2**：LLMPeach 能否探索到语义相关的独特分支？
- **RQ3**：LLMPeach 中的两阶段变异策略表现如何？

4.1 实验设置

4.1.1 被测协议实现

为评估方法的有效性与泛化能力，本文选择了三种具有代表性的网络协议——MQTT[27]、RTSP[28]以及 FTP[29]的五个开源实现作为实验对象。表 4-1列出了被测协议实现的基本信息。

表 4-1 被测协议实现的基本信息

Table 4-1 Basic Information of Tested Protocol Implementations

协议	协议实现	版本	Github 仓库	GitHub Star 数量
MQTT	mosquitto	Commit#28f9147	eclipse-mosquitto/mosquitto	2.6k
MQTT	nanomq	Tag#0.24.12	nanomq/nanomq	2.5k
RTSP	live555[1]	Commit#f4a4e8f	rgaufman/live555	855
FTP	proftpd	Commit#16128ce	proftpd/proftpd	582
FTP	pure-ftpd	Commit#3755156	jedisct1 pure-ftpd	748

这些协议在设计理念、语义结构及应用场景上具有显著差异，能够从多个维度对方法进行系统性验证。

具体而言，在消息格式方面，MQTT 协议采用了由固定报头、可变报头及有效载荷组成的紧凑二进制结构，其消息类型集合相对有限且格式高度规整；RTSP 协议采用了基于回车换行符（CRLF）分隔的纯文本请求/响应格式，包含了中等规模的方法指令集，且支持灵活的头部扩展字段；而 FTP 协议同样基于 ASCII 文本编码，但其拥有极其丰富的命令序列以及海量的三位数字响应状态码。在领域分布方面，MQTT 协议作为轻量化发布/订阅模型的代表，深度植根于对实时性与带宽要求极高的工业物联网与传感器网络领域；RTSP 协议则聚焦于多媒体通信领域，负责处理复杂的实

1 live555 未在源代码托管平台提供官方仓库，本文选取了使用最广泛的镜像仓库。

时音视频流控制与媒体协同；而 FTP 协议作为传统的互联网基础应用层协议，是大规模数据交换与远程文件管理的典型范式。

选取这些协议实现，旨在从不同协议语义结构、消息格式及应用领域等维度，全面评估本方法的适用性与实际表现。

4.1.2 流水线生成的测试组件

表 4-2 展示了各协议实现对应测试组件的基本信息。实验选用 GPT-5.4 作为生成这些组件的大语言模型，它是目前在代码生成与结构化数据理解方面表现最为出色的模型之一。

表 4-2 生成的测试组件的基本信息

Table 4-2 Basic Information of Generated Mutation Components

协议	规范文档页数	数据模型		变异器		修复器	
		消息类型数	代码行数	数量	代码行数	数量	代码行数
MQTT	137	16	456	115	6230	18	1588
RTSP	92	11	587	103	4434	12	1765
FTP	69	34	491	78	6301	10	1088

整体来看，所提出的流水线能够针对不同类型的协议稳定生成完整的测试组件，包括数据模型、变异器和修复器，体现出较好的通用性。从结果分布可以看出，各项指标之间并不存在简单的线性关系，例如规范规模、消息类型数量与最终生成代码规模之间并不严格对应。

例如，RTSP 协议的规范文档页数虽然最少，但却生成了最长的数据模型。这是因为 RTSP 协议具有非常灵活的头部字段定义，需要更细粒度的建模来捕捉其语义特征；而 FTP 协议虽然消息类型数量最多，但生成的测试组件规模却不是最大的。这是因为 FTP 协议虽然有着众多的命令，但每个命令的格式相对简单，通常只有一个参数或者没有参数，因此生成的变异器和修复器的数量相对较少；而 MQTT 协议由于是二进制格式，其结构相对复杂而紧凑，因此生成出了较多的变异器和修复器。

这表明该方法并非依赖表面统计特征进行扩展，而是能够根据协议内部的结构特征与语义复杂度自适应地生成相应组件，从而在不同协议上呈现出差异化的结果。不同协议之间组件规模与复杂度仍存在一定差异，反映出协议设计风格（如结构化程度、命令形式或状态依赖）对生成结果的影响。

4.1.3 基线

由于 LLMPeach 是在 Peach 框架上实现的协议模糊测试工具，选择原有 Peach 框架作为基线进行对比评估是最为合理且具有说服力的。

数据模型的完备性是决定 Peach 模糊测试效果的关键因素。鉴于目标协议的数据模型在 Peach 官方库中并未提供，也没有公开的高质量数据模型可供使用，若直接对比将导致实验结果失去参考价值。

因此，尽管数据模型的生成是大模型智能体流水线中的一个重要环节，但为了确保评估的公平性，在 Peach 框架上也使用了与 LLMPeach 相同的、大模型智能体驱动的测试组件生成流水线生成的高质量数据模型。这样可以确保两者在数据模型质量上的一致性，从而使得评估结果能够更准确地反映出 LLMPeach 在其他测试组件和变异策略设计上的优势。

4.1.4 实验配置

各目标协议的测试组件均通过大模型智能体驱动的流水线生成，涵盖了完整的数据模型、变异器及修复器。随后，上述组件作为插件集成至 LLMPeach 框架中；同时，基线 Peach 框架亦采用相同的数据模型组件。

在模糊测试实验阶段，实验严格遵循 Peach 框架模糊测试的标准流程。针对每个目标协议实现，分别利用 LLMPeach 与基线 Peach 开展对比实验，每组测试持续运行 24 小时，并重复实验 5 次以获取统计平均值。此外，对于同一协议，两种框架均采用完全一致的初始种子文件作为输入。

所有的实验都运行在一台配备 16 核心 Intel(R) Xeon(R) Gold 5320 @2.20 GHz CPU 和 256 GB 内存的服务器上，操作系统为 Ubuntu 24.04.3 LTS。模糊测试框架和被测协议实现均运行在独立且隔离的 Docker 容器中，依靠 Docker 网络进行通信，其中，模糊测试框架的基础镜像均为 `debian:stretch-slim`，被测协议实现的基础镜像均为 `debian:stable-slim`。

4.2 RQ1: 分支覆盖

为了评估 LLMPeach 在协议实现上的模糊测试效果，实验采取分支覆盖作为主要的评估指标。分支覆盖能够反映出测试用例在代码路径上的探索程度，较高的分支覆盖通常意味着测试用例能够触达更多的程序逻辑分支，从而更有可能发现潜在的漏洞。在实验中，被测协议实现首先进行了插桩，随后在模糊测试过程中使用 lcov[30]工具实时收集统计得到了分支覆盖数据。表 4-3展示了两者在分支覆盖上的对比结果。

表 4-3　LLMPeach 与 Peach 在分支覆盖上的对比

Table 4-3　Comparison of Branch Coverage between LLMPeach and Peach

协议实现	**LLMPeach 分支覆盖**	**Peach 分支覆盖**	**提升**	*p*-value	A_{12}
mosquitto	2708.8	2437.2	11.14%	< 0.05	1.00
nanomq	3994.4	3833.6	4.19%	< 0.05	1.00
live555	3148.0	2961.8	6.29%	< 0.05	1.00
proftpd	4643.2	4431.0	4.79%	< 0.05	1.00
pure-ftpd	1026.8	931.4	10.24%	< 0.05	1.00
平均	–	–	**7.33%**	–	–

其中，*p*-value 是通过 Mann-Whitney U 检验[31]计算得到的，用于评估两组数据之间的统计显著性。通常，当 *p*-value 小于 0.05 时，可以认为两组数据之间存在显著差异，即 LLMPeach 在分支覆盖上相较于 Peach 有统计学意义上的提升；A_{12} 是 Vargha-Delaney A 测量值[32]，用于衡量两组数据之间的效应大小。在本实验的意义下，A_{12} 的值范围在 0 到 1 之间，值越接近 1 表示 LLMPeach 在分支覆盖上的表现越优于 Peach，值越接近 0 则表示 Peach 的表现更好。

实验结果表明，在针对五种真实世界协议实现长达 24 小时的测试中，LLMPeach 在所有测试目标上的最终分支覆盖数量均全面超越了基线工具 Peach。总体而言，LLMPeach 实现了平均 7.33% 的分支覆盖数提升。特别是在 mosquitto 和 pure-ftpd 中，提升幅度达到了 11.15% 和 10.23%。所有五个测试组的 p-value 均严格小于 0.05 的显著性水平，证明 LLMPeach 带来的覆盖率增益并非由模糊测试框架底层的随机性导致，而是具有高度的统计显著性。并且，A_{12} 效应量在所有目标协议实现上均达到了 1.00。在统计学意义上，这表明在多轮重复实验中，LLMPeach 针对任一样本的测试表现每一次都绝对优于 Peach。这一结果表明，大语言模型辅助的协议语义感知变异增强方法能够有效提升模糊测试工具在协议实现上的分支覆盖，从而增强漏洞挖掘的能力。

除了最终覆盖数量的静态对比，测试过程中的动态覆盖率增长趋势同样是衡量模糊测试工具效果的一个维度。图 4-1 展示了随着测试时间的推移，LLMPeach 与 Peach 框架在不同协议实现上的分支覆盖变化趋势。其中，X 轴表示测试时间（单位：小时），Y 轴表示分支覆盖数量。图中曲线附近的阴影区域表示该时间点上 5 次实验的标准差范围。

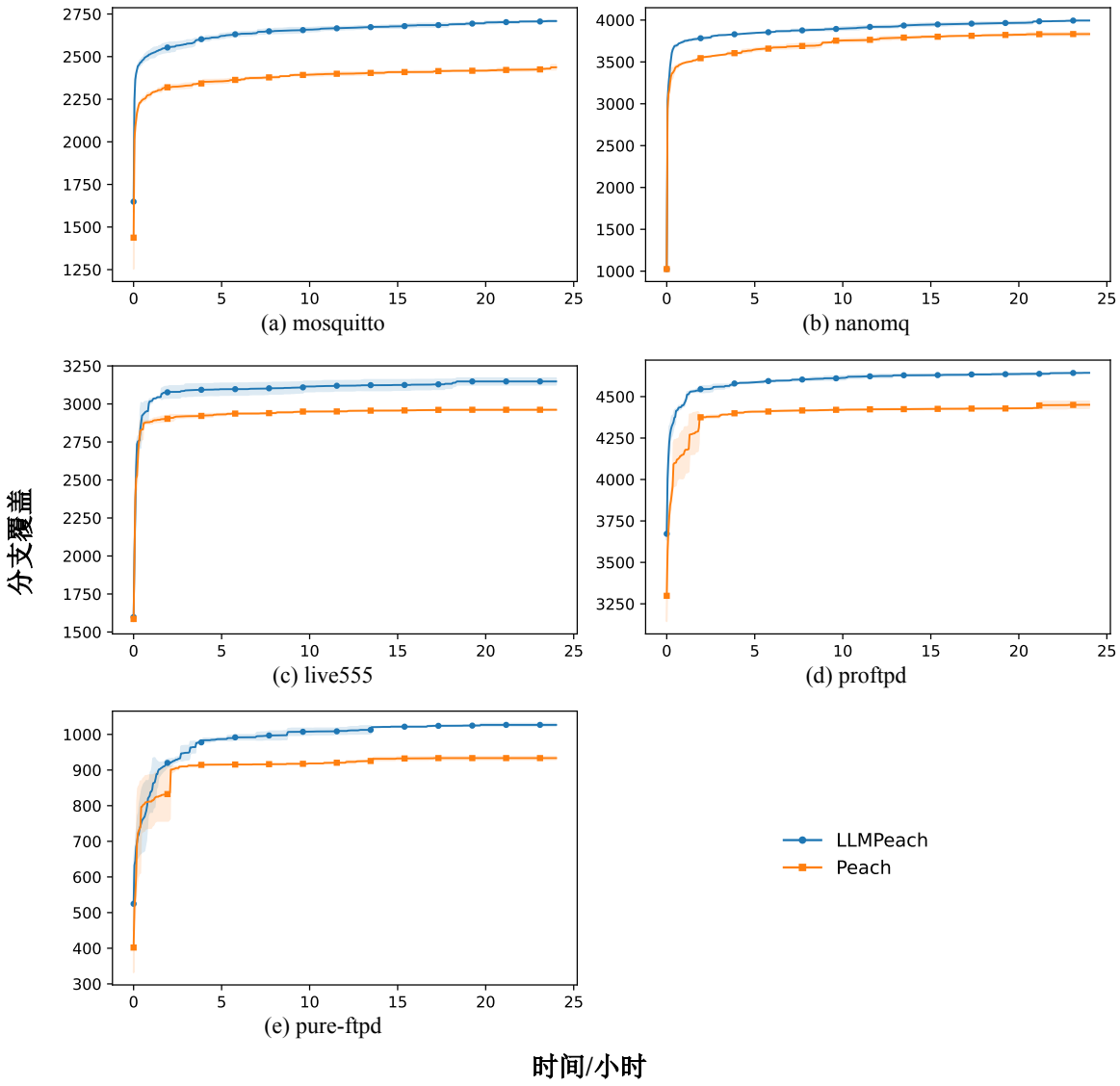


图 4-1　LLMPeach 与 Peach 在分支覆盖上的变化趋势

Figure 4-1　Trend of Branch Coverage between LLMPeach and Peach

观察发现，在测试的初始阶段（前 1-2 小时内），LLMPeach 和 Peach 在目标协议实现的分支覆盖上都呈现出快速增长的趋势，且 LLMPeach 在此阶段分支覆盖数上就已明显领先于 Peach。这表明 LLMPeach 生成的测试组件能够更快地触达协议实现中的关键分支，从而在测试初期就建立起更广泛的覆盖基础。随着测试时间的推移，LLMPeach 和 Peach 的分支覆盖增长速度逐渐趋于平缓，但 LLMPeach 的分支覆盖仍持续保持领先，并且在整个测试过程中都未出现被 Peach 反超的情况。此外，图中表示五次重复实验标准差的阴影区域整体较窄，说明 LLMPeach 在模糊测试中的表现具有较好的稳定性。

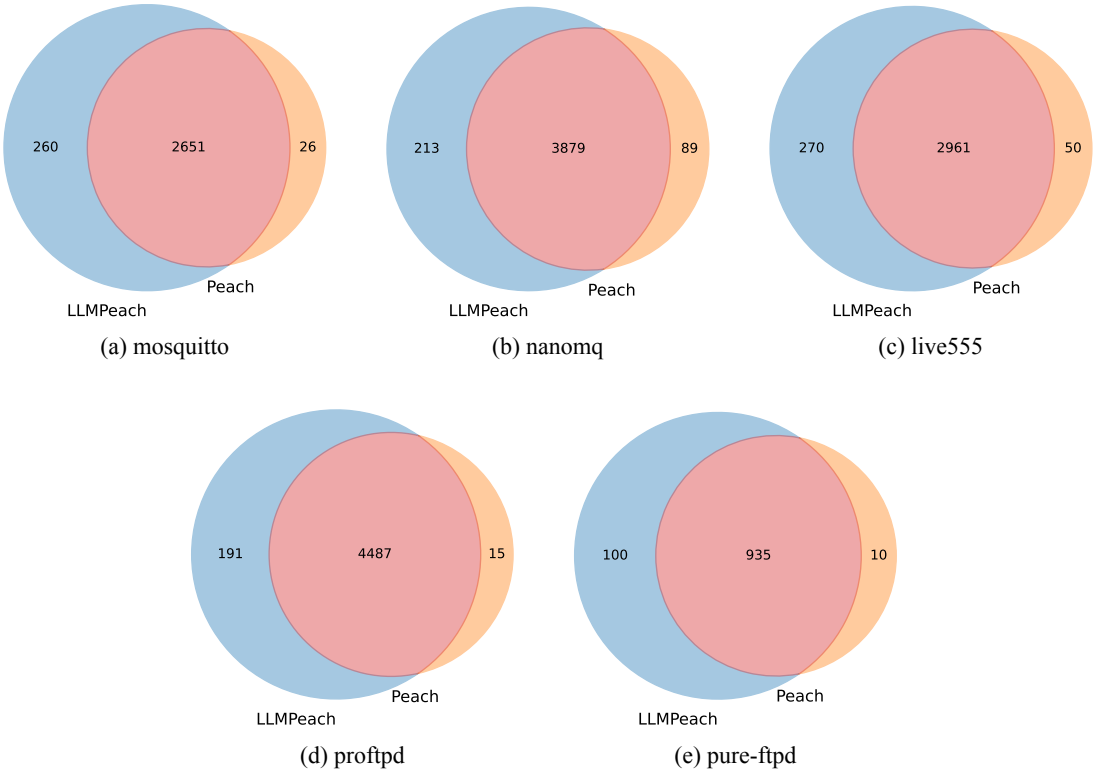
该结果表明，无论是进行短时间的快速测试，还是进行长时间的深度测试，LLM-Peach 都能够持续地提供更高的分支覆盖，从而提升模糊测试的效果。

4.3 RQ2: 独特分支

为了研究 LLMPeach 覆盖到的分支和 Peach 覆盖到的分支之间的关系，评估在多次实验中，LLMPeach 覆盖到的分支中有多少是 Peach 始终未覆盖到的分支，本实验分析了两者在独特分支上的覆盖情况。

独特分支指的是在所有的实验轮次中，一种框架下被覆盖但在另一种框架下未被覆盖的分支。如果某个分支是 LLMPeach 覆盖到的独特分支，意味着该分支在所有 5 次 Peach 测试中都未被覆盖到；反之亦然。LLMPeach 覆盖到的独特分支数量越多，说明 LLMPeach 探索到了更多 Peach 始终未覆盖的分支；而 Peach 覆盖到的独特分支数量越多，则说明 LLMPeach 丢失了更多 Peach 原本能覆盖到的分支。

图 4-2 展示了两者在独特分支上的对比结果，其中每个子图表示一个协议实现，左侧圆表示 LLMPeach 覆盖的分支，右侧圆表示 Peach 覆盖的分支，重叠部分表示两者都覆盖的分支，而非重叠部分则表示各自覆盖到的独特分支。



(a) mosquitto (b) nanomq (c) live555

(d) proftpd (e) pure-ftpd

图 4-2 LLMPeach 与 Peach 在独特分支上的对比

Figure 4-2 Comparison of Unique Branches between LLMPeach and Peach

实验数据显示，在所有被测协议中，LLMPeach 平均触发了 206.8 个独特分支，显著高于 Peach 框架的 38 个。在 mosquitto、proftpd 和 pure-ftpd 中，Peach 的独特分支数量不足 LLMPeach 的 10%；而在 nanomq 和 live555 中，Peach 触发的独特分支数量虽然相对较多，但仍不及 LLMPeach 的 50%。这表明 LLMPeach 确实能够探索到大量

Peach 始终未覆盖的分支，且丢失的原本 Peach 能够覆盖到的分支数量非常有限，其收益远远大于损失。

4.3.1 案例分析

为了探究 LLMPeach 覆盖到的独特分支是否是与协议语义相关的分支，本文对其中一个协议实现 live555 中的独特分支进行了深入的案例分析。该案例来自 `liveMedia/RTSPServer.cpp` 文件中的 `parseTransportHeader` 函数，该函数负责解析 RTSP 请求中的 Transport 头部字段。图 4-3 展示了该函数中的一个代码片段，其中包含了多个条件分支来处理不同的 Transport 头部字段值。

```
1  static void parseTransportHeader(...) {
2    // ...
3    while (sscanf(fields, "%[^;\r\n]", field) == 1) {
4      if (strcmp(field, "RTP/AVP/TCP") == 0) {
5          streamingMode = RTP_TCP;
6      } else if (_strncasecmp(field, "destination=", 12) == 0) {
7          delete[] destinationAddressStr;
8          destinationAddressStr = strDup(field+12);
9      } // ...
10   }
11  // ...
12 }
```

图 4-3　live555 中 parseTransportHeader 函数的代码片段

Figure 4-3　Code Snippet of parseTransportHeader Function in live555

在该函数中，LLMPeach 触发的其中独特分支位于第 4 行和第 6 行，分别对应于对 Transport 头部字段值的不同处理逻辑。分析该函数的实现发现，这些独特分支与 RTSP 协议中 Transport 头部字段的语义相关。例如，第 4 行的条件分支处理了当 Transport 头部字段值为 “RTP/AVP/TCP” 时的情况，这是一种特定的传输模式；而第 6 行的条件分支则处理了当 Transport 头部字段以 “destination=” 开头时的情况，这涉及到目标地址的解析。

在 Pit 文件中，RTSP 请求的 Transport 头部字段的值被建模为一个字符串类型的参数。传统的 Peach 框架只能够知道该参数是一个字符串，但无法理解其内部的语义结构，因此在变异过程中可能会生成一些随机的字符串变异，难以触发上述条件分支；而 LLMPeach 通过大语言模型的语义理解能力，能够识别出该参数的语义特征，并生成一些更具针对性的变异，例如变异为一些符合 RTSP 协议语义的字符串，其中可能就包含了 “RTP/AVP/TCP” 或者以 “destination=” 开头的字符串，从而更有可能触发上述条件分支。

更进一步来说，当 live555 的 `parseTransportHeader` 函数解析 RTSP 请求中的 Transport 头部字段时，如果该字段的值为 “RTP/AVP/TCP”，则会进入第 5 行的条件分支，设置 `streamingMode` 为 RTP_TCP，这会导致后续的媒体流传输采用 RTP over TCP 的方式进行，从而触发更多与该传输模式相关的代码路径。这样，在后续的流程中，LLMPeach 便会触及到更多的独特分支。

通过以上案例分析，更进一步验证了 LLMPeach 覆盖到的独特分支确实是与协议语义相关的分支，这些分支的触发依赖于对协议参数的语义理解和针对性的变异生成，而这些正是 LLMPeach 相较于传统模糊测试工具的核心优势所在。

4.4 RQ3: 两阶段变异策略

为了评估 LLMPeach 中两阶段变异策略的效果，为探究两阶段变异策略对系统性能的贡献，通过对原策略进行替换与修改，构建了以下两种实验变体：

- **LLMPeach**$_{Random}$：将两阶段变异策略替换为传统的随机策略，由随机策略调度 LLM 变异器和 Peach 内置变异器进行变异生成。
- **LLMPeach**$_{LLMOnly}$：仅使用大模型生成的变异器进行变异生成，完全去除 Peach 内置的变异器。

使用以上两个变体对五个协议实现进行模糊测试，采取与之前相同的实验配置，每个工具运行 24 小时，并重复 5 次以获取平均结果。表 4-4展示了两者在分支覆盖上和完整 LLMPeach 的对比。

表 4-4　LLMPeach 与变体在分支覆盖上的对比

Table 4-4　Comparison of Branch Coverage between LLMPeach and Variants

协议实现	LLMPeach 分支覆盖	LLMPeach$_{Random}$		LLMPeach$_{LLMOnly}$	
		分支覆盖	变化	分支覆盖	变化
mosquitto	2708.8	2638.0	-2.61%	2611.6	-3.59%
nanomq	3994.4	3872.2	-3.06%	3755.4	-5.98%
live555	3148.0	3076.4	-2.27%	3026.2	-3.87%
proftpd	4643.2	4604.0	-0.84%	4510.6	-2.86%
pure-ftpd	1026.8	980.8	-4.48%	901.0	-12.25%
平均	–	–	**-2.65%**	–	**-5.71%**

实验结果表明，LLMPeach$_{Random}$ 和 LLMPeach$_{LLMOnly}$ 在分支覆盖上均不如完整的 LLMPeach。LLMPeach$_{Random}$ 相较于 LLMPeach 平均分支覆盖下降了 2.65%，而 LLMPeach$_{LLMOnly}$ 的平均分支覆盖下降幅度更大，达到了 5.71%。

LLMPeach$_{LLMOnly}$ 效果的下降说明，如果仅采用大模型生成的语义相关的变异器，而不结合传统的变异器进行随机的变异探索，整体的分支覆盖效果会显著下降。这表明两阶段变异策略中的第二阶段，在提升分支覆盖方面同样发挥了作用。可能是因为第一阶段生成的语义相关变异器虽然能够触发一些特定的分支，但如果没有第二阶段的 Peach 变异器来提供更多样化的变异输入，测试用例的多样性和覆盖范围就会受到限制，从而导致整体分支覆盖率的下降。这说明两阶段变异策略中保留 Peach 内置变异器是有意义的。

而 LLMPeach$_{Random}$ 的效果下降则说明，如果仅仅加入大模型生成的测试组件，简单地将 LLM 变异器和 Peach 变异器放在一起进行随机调度，而不采用两阶段变异策略来引导变异生成的流程，整体的分支覆盖效果同样会下降。这表明将变异生成过程划分为两个阶段，在第一阶段进行语义相关的变异，第二阶段辅以随机的变异探索，而不是仅仅将两者混合在一起进行简单的扩充，这样的设计是有益于测试效果的。

图 4-4 展示了 LLMPeach 与两个变体在分支覆盖上的变化趋势。

在测试的绝大多数时间段内，LLMPeach 的分支覆盖数量始终领先于两个变体，并在领先以后的时间段内持续保持领先。这表明两阶段变异策略能够在测试的不同阶段持续地提供更高效的变异输入，从而提升分支覆盖率。

此外，还可以发现，如子图 (c)(d)(f) 所示，在一些协议实现上，在测试的前 1-2 小时，LLMPeach$_{LLMOnly}$ 的分支覆盖数量与 LLMPeach 的分支覆盖数量相近，甚至在某些时间点上略微领先于 LLMPeach。这表明在测试的初始阶段，第一阶段的语义相关变异生成策略能够提供一些有针对性的变异输入，从而触发一些特定的分支；但随着测试的进行，第二阶段的随机变异生成策略开始发挥作用，提供更多样化的变异输入，从而使得 LLMPeach 的分支覆盖率逐渐拉开与 LLMPeach$_{LLMOnly}$ 的差距。

此部份实验表明，为大模型生成的语义相关的测试组件设计两阶段变异策略，既能够利用语义相关变异器来触发一些特定的分支，又能够通过随机的变异生成策略来提供更多样化的变异输入，从而在整体上提升分支覆盖率，是一种有效的设计选择。

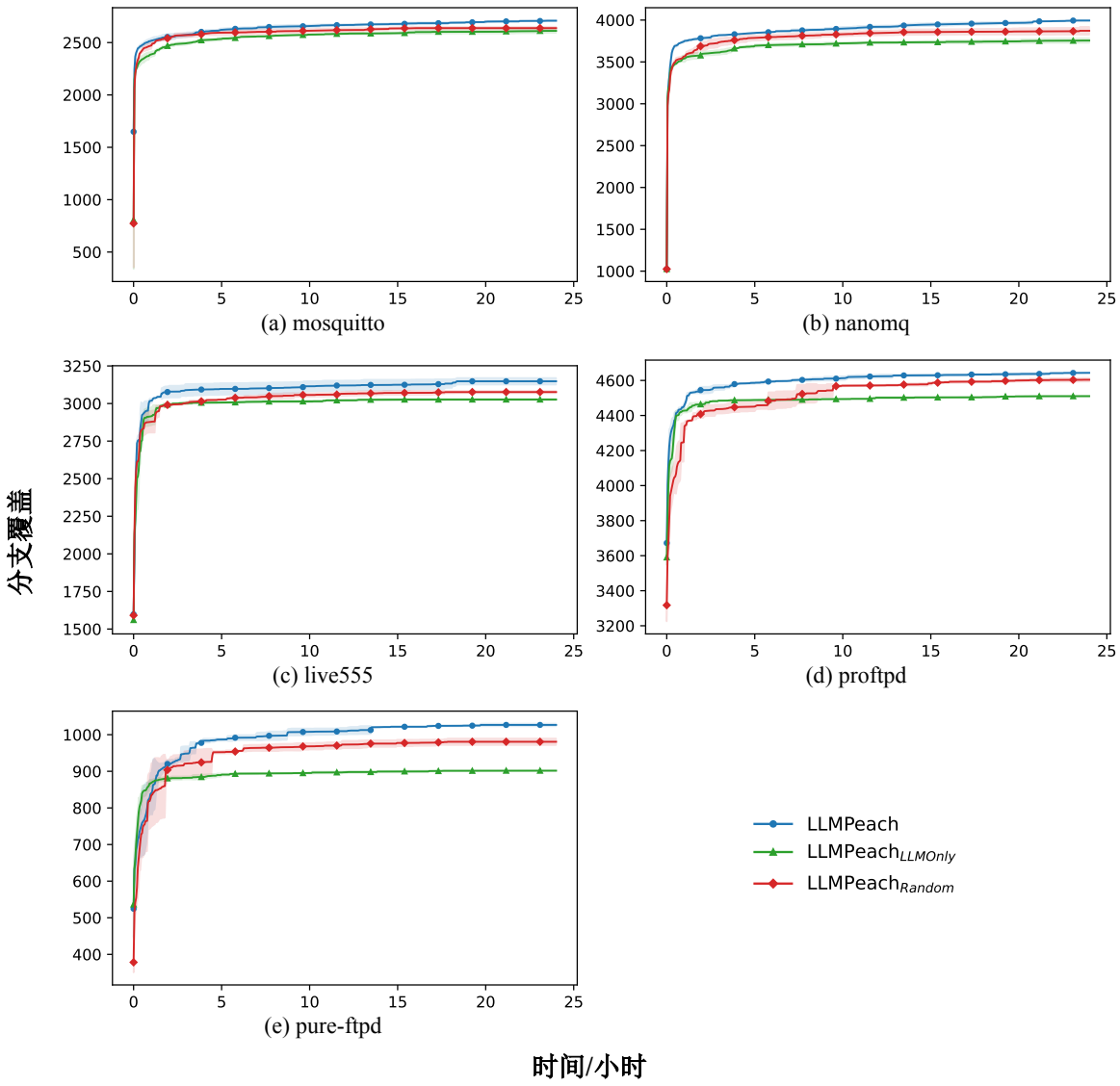


图 4-4　LLMPeach 与变体在分支覆盖上的变化趋势

Figure 4-4　Trend of Branch Coverage between LLMPeach and Variants

5、 总结与展望

针对 Peach 模糊测试框架因缺乏协议语义感知能力而导致的变异盲目、难以触发深层逻辑的问题，本研究提出了一种协议语义感知变异增强方法，并将其集成于 Peach 框架中，开发了 LLMPeach 模糊测试工具。该工具使用由大模型智能体端到端生成的数据模型、变异器和修复器，采用兼顾变异深度与随机性的两阶段变异策略，力求在保持输入结构合理性的同时，更有针对性地对协议字段语义进行探索。

通过在多个协议实现上的实验评估，本研究证实了 LLMPeach 能有效提升被测协议实现的代码分支覆盖率，并能够触及传统 Peach 难以覆盖的深层逻辑路径。这表明结合大语言模型的语义感知机制生成的测试组件能够显著弥补传统模糊测试在语义维度的不足，提升复杂网络协议的漏洞挖掘效率。

尽管 LLMPeach 在协议模糊测试方面取得了初步的成功，但仍存在一些局限性和改进空间。首先，当前的智能体设框架较为简单，未来可以探索更复杂的交互式智能体设计，以提升测试组件生成的质量，并降低开销。其次，当前的测试组件生成中只考虑了数据模型，未来可以进一步引入协议状态机的建模与生成，以实现更为全面的协议语义感知。另外，两阶段变异策略中的参数选择也值得进一步研究和优化。最后，当前的实验评估主要集中在分支覆盖率等静态指标上，未来可以结合漏洞挖掘效率、实际漏洞发现数量等更具实用意义的指标进行评估，以更全面地验证方法的有效性。

参考文献

[1] DE RUITER J, POLL E. Protocol state fuzzing of {TLS} implementations[C]//24th USENIX Security Symposium (USENIX Security 15). 2015: 193-206.

[2] MANÈS V J, HAN H, HAN C, et al. The art, science, and engineering of fuzzing: A survey[J]. IEEE Transactions on Software Engineering, 2019, 47(11): 2312-2331.

[3] DURUMERIC Z, LI F, KASTEN J, et al. The matter of heartbleed[C]//Proceedings of the 2014 conference on internet measurement conference. 2014: 475-488.

[4] 任泽众, 郑晗, 张嘉元, 等. 模糊测试技术综述[J]. 计算机研究与发展, 2021, 58(5): 944-963.

[5] ZHANG X, ZHANG C, LI X, et al. A survey of protocol fuzzing[J]. ACM Computing Surveys, 2024, 57(2): 1-36.

[6] 李伟明, 张爱芳, 刘建财, 等. 网络协议的自动化模糊测试漏洞挖掘方法[J]. 计算机学报, 2011, 34(02): 242-255.

[7] GitLab Inc. Protocol Fuzzer Community Edition[CP/OL]. 2021. https://gitlab.com/gitlab-org/security-products/protocol-fuzzer-ce.

[8] ZHANG Z, ZHANG H, ZHAO J, et al. A survey on the development of network protocol fuzzing techniques[J]. Electronics, 2023, 12(13): 2904.

[9] MENG R, MIRCHEV M, BÖHME M, et al. Large language model guided protocol fuzzing[C]//Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS). 2024.

[10] WANG T, LI Y, PAN Z, et al. LLM-Boofuzz: Generation-Based Black-Box Fuzzing for Network Protocols via LLMs[J]. Electronics, 2025, 14(23): 4550.

[11] PEREYDA J, Contributors. boofuzz: Network Protocol Fuzzing for Humans[CP/OL]. 2015. https://github.com/jtpereyda/boofuzz.

[12] LUO Z, ZUO F, SHEN Y, et al. ICS Protocol Fuzzing: Coverage Guided Packet Crack and Generation[C/OL]//Proceedings of the 57th ACM/IEEE Design Automation Conference (DAC). IEEE, 2020: 1-6. DOI: 10.1145/3370748.3406561.

[13] PHAM V T, BÖHME M, ROYCHOUDHURY A. Aflnet: A greybox fuzzer for network protocols [C]//2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). 2020: 460-465.

[14] BA J, LIANG J, JIANG Y, et al. SGFuzz: Stateful greybox fuzzing for network protocols[C]//Proceedings of the 31st USENIX Security Symposium. 2022.

[15] QIN J, FU K, et al. NSFuzz: A network state-aware fuzzer for network protocols[C]//Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). 2023.

[16] YANG T, et al. BSFuzzer: Context-Aware Semantic Fuzzing for BLE Logic Flaw Detection[C]// Proceedings of the Network and Distributed System Security (NDSS) Symposium 2026. 2026.

[17] CHENG M, ZHU K, CHEN Y, et al. Msfuzz: Augmenting protocol fuzzing with message syntax comprehension via large language models[J]. Electronics, 2024, 13(13): 2632.

[18] LIAN Y, HU Z. Smarter Peach: Add Eyes to Peach Fuzzer[C/OL]//Proceedings of RootedCon. Madrid, Spain, 2017. https://www.rootedcon.com/.

[19] TANENBAUM A S, WETHERALL D J. Computer Networks[M]. 5th. Prentice Hall, 2011.

[20] MILLER B P, FREDRIKSEN L, SO B. An empirical study of the reliability of UNIX utilities[J]. Communications of the ACM, 1990, 33(12): 32-44.

[21] LIANG H, PEI X, JIA X, et al. Fuzzing: State of the art[J]. IEEE Transactions on Reliability, 2018, 67(3): 1199-1218.

[22] GODEFROID P, LEVIN M Y, MOLNAR D A, et al. Automated whitebox fuzz testing.[C]//Ndss: vol. 8. 2008: 151-166.

[23] ZALEWSKI M. American fuzzy lop[CP/OL]. 2014. http://lcamtuf.coredump.cx/afl.

[24] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C] //International Conference on Learning Representations (ICLR). 2023.

[25] SCHICK T, DWIVEDI-YU J, DESSÌ R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[J]. arXiv preprint arXiv:2302.04761, 2023.

[26] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems: vol. 33. 2020: 9459-9474.

[27] OASIS Standard. MQTT Version 3.1.1[Z]. http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html. 2014.

[28] SCHULZRINNE H, RAO A, LANPHIER R. Real Time Streaming Protocol (RTSP): 2326[R/OL]. IETF. 1998. https://rfc-editor.org/rfc/rfc2326.txt.

[29] POSTEL J, REYNOLDS J. File Transfer Protocol (FTP): 959[R/OL]. IETF. 1985. https://rfc-editor.org/rfc/rfc959.txt.

[30] LTP Team. LCOV: A Graphical Front-end for GCOV[Z]. https://github.com/linux-test-project/lcov. Accessed: 2026-04-19. 2002.

[31] MANN H B, WHITNEY D R. On a test of whether one of two random variables is stochastically larger than the other[J]. The Annals of Mathematical Statistics, 1947: 50-60.

[32] VARGHA A, DELANEY H D. A critique and improvement of the CL common language effect size statistics of McGraw and Wong[J]. Journal of Educational and Behavioral Statistics, 2000, 25(2): 101-132.

附录

本研究的全部代码可以在 https://github.com/llipengda/llm-peach-artifacts 获取。该代码库包含了 LLMPeach 模糊测试框架的实现、大模型智能体驱动的测试组件生成流水线实现以及实验评估与分析的相关脚本。

致谢

行文至此，本文的撰写工作已经接近尾声，我的本科学习生涯也即将画上一个句号。在此，我想借此机会向所有在这段旅程中给予我支持和帮助的人表达我最诚挚的感谢。

首先，我要感谢华东师范大学。这所学校不仅提供了优质的教育资源和学术环境，更是培养了我独立思考和创新能力的摇篮。在这里，我遇到了许多优秀的老师和同学，他们的智慧和热情激励着我不断前行。在这四年的学习期间，华师大的人文素养也深深地感染了我，一些极为优秀的通识课程让我收获良多，以至于时常回忆起这些课程时，不免感叹当时没有好好珍惜。

我要特别感谢我的导师，苏亭教授。这篇论文的完成离不开苏教授的悉心指导。苏教授不仅在学术上给予了我无私的指导和支持，还时常关心我的成长和发展。他严谨治学态度和对学生的关怀让我深受感动，也激励着我不断追求卓越，非常期待在未来的学习和研究中继续得到他的指导和帮助。

我要感谢王可以学姐。她在这篇论文的撰写过程中给予了我极大的帮助和支持。在研究方法与实验评估的设计以及最后的论文撰写过程中，学姐都提供了宝贵的建议和指导，并提出了很多修改意见。她的专业知识和耐心帮助我克服了许多困难，使得这篇论文能够顺利完成。

感谢符俊哲学长和蔡耀德学长，他们在方法设计和实验上提供了很多帮助，与他们的讨论和交流让我受益很多；感谢周炽金老师、孙海英老师和万成城老师。周老师、孙老师和万老师每次都来参加我们的讨论，为我提供了很多珍贵的建议；感谢上海工业控制安全创新科技有限公司的张绍景和柳泽，与他们的交流为我提供了一些新的思路。

在毕业论文之外，我要感谢的人还有很多——

感谢可爱的同学们，与你们的相处让我在大学生活中收获了许多美好的回忆。或许有一天，我会回想起这些时光，感慨万千；感谢敬爱的老师们，你们的教诲和指导让我受益匪浅。无论是在课堂上还是在课外，你们的耐心和专业精神都深深地影响了我，激励着我不断追求知识和成长。

感谢我的舍友们，与你们的相处很愉快，虽然在一起的时间不算长，但实在感谢你们的陪伴和支持。当我写下这句话时，已是深夜，所以要更加感谢你们的包容。

感谢我的朋友们，无论你们现在在哪里，将来又将去往何方。在这段旅程中，你们给予了我无尽的支持和陪伴，让我的生活丰富多彩。其中，在大学认识的新朋友中，我要尤其感谢武泽恺。和你的深度交流、讨论，让我思考了许多，感谢你总是愿意听我分享一些奇奇怪怪的想法，感谢你的陪伴，这份友谊是我大学生活中最宝贵的财富之一。我还要感谢罗剑波，和你总能进行一些有意义的交流。

最后，我要感谢我的家人。你们的无私支持和鼓励是我能够坚持到现在的最大动力。无论是在学业上还是生活中，你们一直是我最坚强的后盾。感谢你们一直以来的理解和支持，我会继续努力，不辜负你们的期望。

沈从文写到：“凡事都有偶然的凑巧，结果却又如宿命的必然。”我时常想起这句话，回顾我的大学生活，确实有很多偶然的相遇和经历，但这些偶然的事件却又构成了我人生中必然的轨迹。正是这一个又一个偶然，让我成长为今天的自己。我感恩这一路上的点点滴滴，无论是成功还是失败，无论是喜悦还是忧伤，无论是刻骨铭心的经历还是平凡的日常。